

Curs 1	4
Curs 2	4
Criptanaliza, criptografie, criptologie = criptanaliza + criptografie	4
Tipuri de atac	5
Sisteme de criptare istorice	5
Criptografie moderna:	5
Securitate perfectă	5
Un exemplu de cifru sigur - One Time Pad (OTP)	6
Curs 3	6
Securitate computațională ->	6
Neglijabil și ne-neglijabil	6
Criptarea simetrică - redefinită	7
PRG - Pseudo Random Number Generator	7
Sistem de criptare bazat pe PRG	8
Curs 4.0	8
Securitate - interceptare simplă	8
Securitate - interceptare multiplă	8
Securitate CPA	9
4.1 Informatii extra PRG	9
PRG în practică	10
Sisteme fluide	10
Sisteme fluide vs OTP	10
Securitate la interceptare multiplă	10
Proprietati ale PRG in modul nesincronizat	10
4.2 Linear-Feedback Shift Registers (LFSR)	10
RC4	11
Trivium	11
Curs 4.3 - Criptografia simetrică, ECB, CBC, OFB, CTR, PRP, PRF	12
Sisteme de criptare fluide vs bloc	12
PRP	12
PRF	12
Moduri de utilizare - ECB/CBC/OFB/CTR	13
Curs 5.1- Scheme de criptare CPA-sigure bazate pe PRF	14
Curs 5.2 - Securitate CCA	14
Curs 5.3 - Construcții practice PRF, AES, Feistel	16
Paradigma confuzie-difuzie	16
Rețele de substituție - permutare	16
AES	17
Rețele Feistel	17
Curs 6.1 - Padding-oracle attack	18
Atacul bazat pe oracol de padding	18
CBC cu padding PKCS7	18
Ideea atacului cu oracol de padding	18
Curs 6.3 - Data Encryption Standard (DES)	20
Descriere DES	20
Criptanaliză avansată	21
Criptanaliza diferențială	21
Criptanaliza liniară	22
Atacul meet-in-the-middle	22
Triplu-DES (3DES)	23
Curs 7.1 - MAC	23
Criptare vs autentificarea mesajelor	23
Coduri de autentificare a mesajelor - MAC	24

Cazuri de folosire ale MAC	24
Securitate MAC	24
Construcția MAC-urilor sigure	25
MAC-uri pentru mesaje de lungime variabilă	25
CBC-MAC	25
HMAC	26
Securitate CCA vs Criptare autenticată	27
Criptare autenticată în practică	27
7.2 - Funcții Hash	27
Atacul zilei de naștere	29
Transformarea Merkle-Damgård	29
Curs 8.1 - SHA-3	30
Keccak	30
8.2 - Aplicații ale funcțiilor hash	31
Stocarea parolelor	31
Atac de tip dicționar	31
Salting	32
8.3 - Criptografie cu cheie publică (asimetrică)	33
Sisteme de criptare asimetrice	33
8.5 - Prezumții criptografice dificile	35
Problema factorizării	35
Generarea numerelor prime	35
Testarea primalității	36
Algoritmi de factorizare	36
Problema logaritmului discret	36
Curs 9.1 - Noțiuni de securitate în criptografia asimetrică	37
Criptografia asimetrică	37
Securitate pentru interceptare simplă	37
Insecuritatea schemelor deterministe	37
Securitate CCA	38
Curs 9.2 - Criptare hibridă	38
Curs 9.3 - RSA	38
GenRSA	39
Textbook RSA	40
Curs 9.4 - PKCS	40
Padded RSA	40
PKCS	41
OAEP	41
Utilizarea RSA în practică	41
Curs 10.2 - Problema logaritmului discret (PLD)	42
PLD - din curs 8.5	42
Algoritmi pentru calculul logaritmului discret	42
Metoda Baby-step/Giant-step - algoritmi generici	42
Algoritmi non-generici pentru calculul logaritmului discret	43
Curs 10.3 - Schimbul de chei Diffie-Hellman	43
Conceptul de schimb de chei	43
Schimbul de chei Diffie-Hellman	43
Corectitudinea protocolului	43
Computational Diffie-Hellman (CDH)	44
Decisional Diffie-Hellman (DDH)	44
Atacul Man-in-the-Middle (MITM)	44
10.4 Sistemul de criptare ElGamal	45
Probleme, întrebări legate de ElGamal	45
Curs 10.5 - Criptografia bazată pe Curbe Eliptice	46
Exemplu adunare puncte $P + Q \rightarrow$	46

Curbe eliptice folosite in practica	46
• Teorema lui Hasse	46
Curbe eliptice standardizate, folosite in practica, sigure si cu implementari eficiente	47
ECDLP - Elliptic Curve Discrete Logarithm Problem	47
Securitate ECDLP	47
Criptografia pe Curbe Eliptice(Elliptic Curve Cryptography)	47
Security level	47
Important de retinut	47
Curs 10.6 Schimbul de chei Diffie-Hellman si ElGamal pe Curbe Eliptice	48
Diffie-Hellman pe curbe eliptice	48
Corectitudine	48
Securitate	48
ECCDH - Elliptic Curve Computational Diffie-Hellman	48
ECDDH - Elliptic Curve Decisional Diffie-Hellman	49
Atacul Man-in-the-Middle pe Diffie-Hellman pe curbe eliptice	49
ElGamal pe Curbe Eliptice	49
Sistemul de criptare	49
Securitate	49
Important de retinut	50
Curs 11: Semnaturi digitale si PKI	50
Aplicatii ale semnaturilor digitale	50
Avantaje semnaturi digitale fata de MAC-uri	50
Atacuri prin replicare	52
Constructie de scheme de semnatura digitala	52
Semnatura digitala bazata pe RSA	52
Schema de identificare Schnorr	53
DSA/ECDSA	54
Certificate si PKI	54
Modelul "web of trust"	55
Invalidarea certificatelor	55
Curs 12: Criptografia post-cuantica	56
Criptografia post-cuantica vs criptografia cuantica	56
Criptografia simetrica post-cuantica	56
Algoritmul lui Shor si impactul lui asupra criptografiei asimetrice	57
Sisteme de criptare cu cheie publica post-cuantice	57
Problema LWE - Learning With Errors	58
Curs 13 - TLS (Transport Layer Security)	59
Alte informatii TLS 1.3	60
TLS 1.3 vs 1.2	60

Curs 1

Amenințări și obiectivele securității

Triada C-I-A (confidentiality, integrity, availability)

1. Confidentialitate – Bunurile protejate pot fi văzute doar de persoanele autorizate
2. Integritate - Bunurile protejate pot fi modificate doar de persoanele autorizate
3. Disponibilitate (availability) -Bunurile protejate pot fi utilizate doar de persoanele autorizate

Principiile securității criptografice

1. Principiul lui Kerkoff: doar cheia este secretă, construcția (algoritmul) e publică
2. Principiul separării cheilor: chei diferite pentru scopuri diferite
3. Principiul diversității: folosește tipuri diferite de algoritmi

Alte principii

1. Principiul simplității: keep it simple
2. Securitate implicită (security by default):trebuie gândită de la început, iar nu adăugată mai târziu
3. Principiul încrederii minime (principle of minimal trust): minimizarea numărului de entități cărora le acordăm încredere
4. Principiul celei mai slabe verigi (principle of the weakest link): securitatea unui sistem este dată de punctul sau cel mai slab
5. Principiul celui mai mic privilegiu (least privilege): se acordă exact privilegiul necesar pentru efectuarea unei activități

Principiile securității

1. Principiul modularității: totul trebuie păstrat modular
2. Defence in depth – securitate la diverse nivele

Curs 2

- Sistem de criptare simetric: cele istorice(Rail Fence, Cezar, Vinegere), securitatea perfectă(OTP)

Un sistem de criptare simetric definit peste (K, M, C) cu K = spațiu cheilor, M = spațiul textelor clare, C = spațiul textelor criptate este un triplet (Gen, Enc, Dec) , unde:

Gen = alg. Probabilistic de generare a cheilor care întoarce o cheie k conform unei distribuții

$Enc : K \times M \rightarrow C$ $Dec : K \times C \rightarrow M$ a.i. $\forall m \in M, k \in K : Dec_k(Enc_k(m)) = m$.

K = variabila aleatoare ce reprezintă valoarea cheii returnată de Gen .

$Pr[K = k]$ = probabilitatea cheii generate de Gen de a lua valoarea k , $\forall k \in K$.

M = variabila aleatoare ce reprezintă mesajul care se criptează.

$Pr[M = m]$ = probabilitatea ca mesajul criptat să ia valoarea $m \in M$.

Ex de probabilitate de distribuție peste M :

$M = \{\text{atacați azi, nu atacați}\}$, cu $Pr[M = \text{atacați azi}] = 0.2$ și $Pr[M = \text{nu atacați}] = 0.8$

Criptanaliza, criptografie, criptologie = criptanaliza + criptografie

Def criptanaliza = procesul de determinare a cheii aferente unui sistem de criptare, cunoscând doar textul criptat.

Def criptografie = o ramura a matematicii care se ocupa cu securizarea informatiei precum si cu autentificarea si restrictionarea accesului intr-un sistem informatic

Tipuri de atac

-atac cu text criptat (ciphertext-only attack: Atacatorul stie doar textul criptat - poate incerca brute force => (atacator pasiv)
-atac cu text clar: Atacatorul cunoaste una sau mai multe perechi (text clar, text criptat) => (atacator pasiv)
-atac cu text clar ales (chosen-plaintext attack - CPA): Atacatorul poate obtine criptarea unor texte clare alese => (atacator activ)
-atac cu text criptat ales (chosen-ciphertext attack - CCA): Atacatorul are posibilitatea sa obtina decriptarea unor texte alese de el. => (atacator activ)
Atacator activ - poate cere decriptarea anumitor mesaje si, pe baza acestora, cere decriptarea unor texte in mod adaptiv

Sisteme de criptare istorice

Cifruri de permutare / transpozitie = rearanjarea literelor in textul clar pentru a obtine textul criptat.

Rail Fence: exemplu: Text clar: mesaj criptat, Cheia $k = 3$, Text criptat: MARTEJIASCPT

M A R T

E J I A

S C P T

Cifruri de substitutie monoalfabetice: cifrul lui Cezar, substitutie simpla, sistemul Cavalerilor de Malta.

Shift cipher = A shift cipher involves replacing each letter in the message by a letter that is some fixed number of positions further along in the alphabet.

Principiul cheilor suficiente = O schema sigura de criptare trebuie sa aiba un spatiu al cheilor suficient de mare a.i. Sa nu fie vulnerabila la cautarea exhaustiva.

Analiza de frecventa = determinarea corespondentei intre alfabetul clar si alfabetul criptat pe baza frecventei de aparitie a literelor in text, cunoscand distributia literelor in limba textului clar.

Cifruri de substitutie polialfabetice / poligrafice : sistemul Playfair, sistemul Vigenere.

Sistemul Vigenere:

Ex: attackatdawn -> LEMONLEMONLE, apoi corespondenta din tabelu ala cu litere.

Criptografie moderna:

Principiul 1 - orice problema criptografica necesita o definitie clasica si riguroasa.

Principiul 2 - securitatea primitivelor criptografice se bazeaza pe prezumptii clare de securitate.

Principiul 3 - orice constructie criptografica trebuie sa fie insotita de o demonstratie de securitate conform principiilor anterioare.

Securitate perfectă

O schemă de criptare peste un spațiu al mesajelor M este perfect sigură dacă pentru orice probabilitate de distribuție peste M , pentru orice mesaj $m \in M$ și orice text criptat c pentru care $\Pr[C = c] > 0$, următoarea egalitate este îndeplinită: $\Pr[M = m|C = c] = \Pr[M = m]$

- $\Pr[M = m]$ - probabilitatea a priori ca Alice să aleagă mesajul m ;

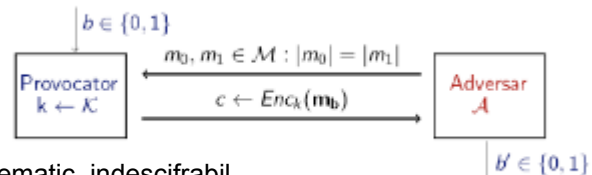
- $\Pr[M = m|C = c]$ - probabilitatea a posteriori ca Alice să aleagă mesajul m , chiar dacă textul criptat c a fost văzut

Un exemplu de cifru sigur - One Time Pad (OTP)

- **Perfect sigur**
- Criptare: $c = m \text{ XOR } k$
- Decriptare: $m = k \text{ XOR } c$
- Avantaje: - foarte rapid la criptare si decriptare
- Dezavantaje: - lungimea cheii e cel puțin egala cu lungimea mesajului
- cheia trebuie folosita o singura data
- Concluzie: nu poate fi utilizat

Curs 3

Securitate computațională ->



Ideea de bază: principiul 1 al lui Kerckhoffs

Un cifru trebuie să fie practic, dacă nu matematic, indescifrabil.

Sunt de interes mai mare schemele care practic nu pot fi sparte deși nu beneficiază de securitate perfectă;

1. Adversari limitați computațional/eficienti/timp polinomial

Exemplu: Un atacator care realizează un atac prin forță brută peste spațiul cheilor și testează o cheie/ciclu de ceas

2. Adversarii pot efectua un atac cu succes cu o probabilitate foarte mică

Pentru securitatea perfectă am dat două definiții echivalente, a doua subliniază ideea de

indistinctibilitate: adversarul nu poate distinge între criptările a două mesaje diferite

Vom defini indistinctibilitatea pe baza unui experiment $\text{Priv}_{A, \pi}^{\text{eav}}(n)$ unde $\pi = (\text{Enc}, \text{Dec})$ este schema de criptare

Personaje participante: adversarul A care încearcă să spargă schema și un provocator (challenger).

Trebuie să definim capacitățile adversarului: el poate vedea un singur text criptat cu o anumită cheie, fiind un adversar pasiv care poate rula atacuri în timp polinomial, și nu are nici o altă interacțiune cu Alice sau Bob

O schema de criptare este (t, ϵ) -indistinctibilă dacă orice adversar care rulează în timp cel mult t

$$\Pr[\text{Priv}_{A, \pi}^{\text{eav}} = 1] \leq \frac{1}{2} + \epsilon \quad \text{probabilitatea de succes a adversarului} \leq \epsilon, \text{ adversarul rulează în timp } \leq t$$

O schema de criptare este indistinctibilă dacă pentru orice adversar PPT, există o funcție neglijabilă ϵ așa încât

$$\Pr[\text{Priv}_{A, \pi}^{\text{eav}}(n) = 1] \leq \frac{1}{2} + \epsilon(n)$$

Neglijabil și ne-neglijabil

În practică: ϵ este scalar și ϵ ne-neglijabil dacă $\epsilon \geq \frac{1}{2^{30}}$, ϵ neglijabil dacă $\epsilon \leq \frac{1}{2^{80}}$

În teorie: ϵ este funcție $\epsilon : \mathbb{Z}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ și $p(n)$ este o funcție polinomială în n (ex.: $p(n) = nd$, d constantă) ϵ ne-neglijabil în n dacă $\exists p(n) : \epsilon(n) > 1/p(n)$, ϵ neglijabil în n dacă $\forall p(n), \exists nd \text{ a.î. } \forall n \geq nd : \epsilon(n) \leq 1/p(n)$

Valori concrete pentru n

Presupunem că pentru o schema de criptare concretă, un adversar care rulează în timp n^3 minute reușește să spargă schema cu probabilitate $2^{40} * 2^{-n}$.

Criptarea simetrică - redefinită

Definiție

Un sistem de criptare simetric definit peste (K, M, C) , cu: K = spațiul cheilor, M = spațiul textelor clare (mesaje), C = spațiul textelor criptate este un triplet (Gen, Enc, Dec) , unde:

1. $Gen(1^n)$: este algoritmul probabilistic de generare a cheilor care întoarce o cheie k conform unei distribuții
2. Enc : primește o cheie k și un mesaj $m \in \{0, 1\}^*$ și întoarce $c \leftarrow Enc_k(m)$
3. Dec : primește cheia k și textul criptat și întoarce m sau "eroare".

Aleatorism

Încercăm criptare în stil OTP: cheia va masca mesajul dar

- masca nu va fi doar cheia ci $mask = f(key)$ unde f este o funcție de extindere a cheii
- pentru securitate perfectă masca trebuie să fie perfect aleatoare
- pentru securitate computațională, este suficient ca masca să apară aleatoare pentru un adversar PPT chiar dacă nu este
- Vom avea nevoie întâi să definim noțiunea de generatoare de numere pseudoaleatoare ca element important de construcție pentru schemele de criptare simetrice

Un sir pseudoaleator "arăta" similar unui sir uniform aleator din punct de vedere al oricărui algoritm polinomial

PRG - Pseudo Random Number Generator

- Acesta este un algoritm determinist care primește o "samantă" relativ scurtă s (seed) și generează o secvență pseudoaleatoare de biți
- Notăm $|s| = n$, $|PRG(s)| = l(n)$
- PRG prezintă interes dacă: $l(n) \geq n$ (altfel NU "generează aleatorism")

Definiție

Fie $l(\cdot)$ un polinom și G un algoritm polinomial determinist a.i. $\forall n \in \{0, 1\}^n$, G generează o secvență de lungime $l(n)$. G se numește generator de numere pseudoaleatoare (PRG) dacă se satisfac 2 proprietăți:

1. Expansiune: $\forall n, l(n) \geq n$

2. Pseudoaleatorism: $\forall$ algoritm PPT D , $\exists$ o funcție neglijabilă $negl$ a.i.:

$|\Pr[D(r) = 1] - \Pr[D(G(s)) = 1]| \leq negl(n)$ unde $r \leftarrow_R \{0, 1\}^{l(n)}$, $s \leftarrow_R \{0, 1\}^n$ $l(n)$ se numește factorul de expansiune al lui G

Notatii

D = Distinguisher, PPT = Probabilistic Polynomial Time, $x \leftarrow_R X$ = x este ales uniform aleator din X , $negl(n)$ = o funcție neglijabilă în (parametrul de securitate) n . În plus vom nota A un adversar (Oscar / Eve), care (în general) are putere polinomială de calcul

Observații

Distribuția output-ului unui PRG este departe de a fi uniformă

Exemplificăm pentru un G care dublează lungimea intrării i.e. $l(n) = 2n$

Pentru distribuția uniformă peste $\{0, 1\}^{2n}$, fiecare din cele 2^{2n} este ales cu probabilitate $1/2^{2n}$

Considerăm distribuția output-ului lui G când primește la intrare un sir uniform de lungime n

Numărul de siruri diferite din codomeniul lui G este cel mult 2^n

Probabilitatea ca un sir de lungime $2n$ să fie output al lui G este $2^n/2^{2n} = 1/2^n$

Seed-ul unui PRG este analog cheii unui sistem de criptare

Seed-ul trebuie ales uniform și menținut secret

Seed-ul trebuie să fie suficient de lung așa încât un atac prin forță brută să nu fie fezabil

Sistem de criptare bazat pe PRG

Definiție

Un sistem de criptare (Enc, Dec) definit peste (K, M, C) se numește sistem de criptare bazat pe PRG dacă:

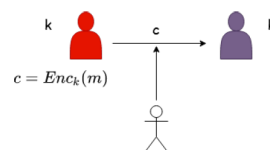
1. $\text{Enc} : K \times M \rightarrow C$ $c = \text{Enc}_k(m) = G(k) \oplus m$
2. $\text{Dec} : K \times C \rightarrow M$ $m = \text{Dec}_k(c) = G(k) \oplus c$ unde G este un generator de numere pseudoaleatoare cu factorul de expansiune l , $k \in \{0, 1\}^n$, $m \in \{0, 1\}^{l(n)}$

Teorema

Dața G este PRG, atunci sistemul definit anterior este un sistem de criptare simetric de lungime fixa computationally sigur pentru un atacator pasiv care poate intercepta un mesaj.

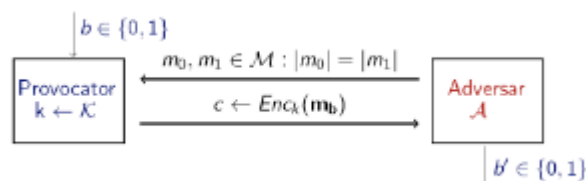
Curs 4.0

Reamintim noțiunea de indistinctibilitate definită anterior, în cazul unui adversar care interceptează un singur mesaj criptat



Experimentul $\text{Priv}_{\mathcal{A}, \pi}^{\text{eav}}(n)$

Output-ul experimentului este 1 dacă $b' = b$ și 0 altfel. Dacă $\text{Priv}(n) = 1$ spunem că adversarul a efectuat experimentul cu succes.

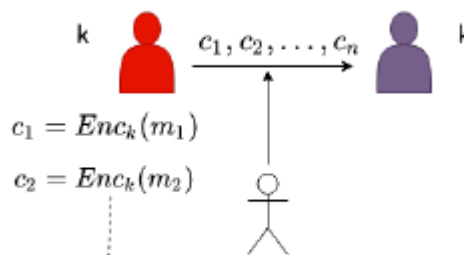


Securitate - interceptare simplă

O schemă de criptare $\pi = (\text{Enc}, \text{Dec})$ este indistinctibilă în prezența unui atacator pasiv dacă pentru orice adversar A există o funcție neglijabilă negl așa încât $\Pr[\text{Priv}_{\mathcal{A}, \pi}^{\text{eav}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$.

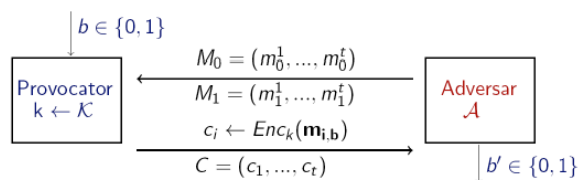
Securitate - interceptare multiplă

- În definiția precedentă am considerat cazul unui adversar care primește **un singur** text criptat
- În realitate, în cadrul unei comunicații se trimit **multiple mesaje** pe care A le poate intercepta



Experimentul $\text{Priv}_{\mathcal{A}, \pi}^{\text{mult}}(n)$

- Outputul experimentului este 1 dacă $b' = b$, sau 0 altfel;
- Definiția de securitate este aceeași doar că se referă la experimentul de mai sus
- Securitatea pentru interceptare **simplă** nu implică și securitate pentru interceptare **multiplă**

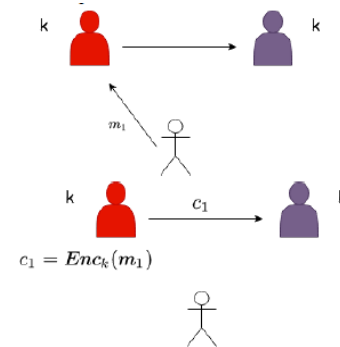
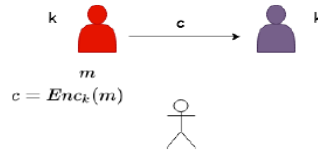


Teoremă - O schemă de criptare (Enc, Dec) unde Enc este determinist nu are proprietatea de securitate la interceptare multiplă

- Pentru a obține sec la intercept multiplă avem nevoie de o schemă de criptare probabilistă a.î. La criptări succesive ale aceluiași mesaj să obținem texte criptate diferite

Securitate CPA

- CPA - **chosen plaintext attack** - adversarul poate obține criptarea unor mesaje alese de el
- Acesta poate cere criptarea unor mesaje alese de el repetitiv (polinomial de multe ori)
- Ulterior, acesta observă criptarea unui mesaj necunoscut:
- Dorim ca acesta să nu poată afla niciun fel de informație despre mesajul m

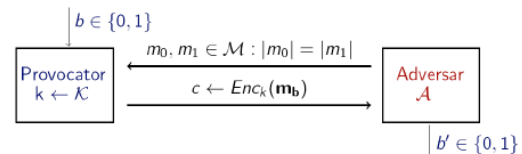


Capabilități adversar:

- poate interacționa cu un **oracol de criptare** fiind un adversar *activ* care poate rula atacuri în timp polinomial.
- Poate trimite către oracol orice mesaj m și primi înapoi criptarea acestuia
- Dacă sistemul este nedeterminist atunci oracolul folosește de fiecare dată o valoare aleatoare nouă, neutilizată anterior

Experimentul $\text{Priv}_{A,\pi}^{\text{cpa}}(n)$

- Considerăm securitatea impactată dacă adversarul poate distinge 2 mesaje aleatoare
- Experiment de indistinguibilitate: $\text{Priv}_{A,\pi}^{\text{cpa}}(n)$ unde $\pi = (\text{Enc}, \text{Dec})$ și n este parametrul de securitate al schemei π
- Persoanele participante: adversarul A și un provocator (challenger)
- Output-ul experimentului este 1 dacă $b' = b$ și 0 altfel. Dacă outputul este 1 atunci A a efectuat experimentul cu succes



Schemă CPA-sigură

O schemă de criptare $\pi = (\text{Enc}, \text{Dec})$ este **CPA-sigură** dacă $\Pr[\text{Priv}_{A,\pi}^{\text{cpa}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$.

CPA-sigură dacă pentru orice adversar PPT A există o funcție de neglijabilă așa încât Cu alte cuvinte, un adversar nu poate determina care text clar a fost criptat cu o probabilitate semnificativ mai mare decât dacă ar fi ghicit - dat cu banul (chiar dacă are acces la oracol).

Proprietati

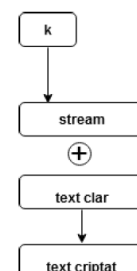
- Un sistem de criptare CPA-sigur are întotdeauna proprietatea de indistinguibilitate
 - Experimentul $\text{Priv}_{A,\pi}^{\text{cpa}}(n)$ este $\text{Priv}_{A,\pi}^{\text{cpa}}(n)$ în care A nu folosește oracolul
- Un sistem de criptare **determinist nu poate fi CPA-sigur**

Securitate CPA la criptare multiplă

Securitatea pentru criptare **simplică** implică securitatea pentru criptare **multiplă**. Experimentul este la fel ca mai sus doar că se considera 2 mulțimi de mesaje în loc de 2 mesaje.

4.1 Informații extra PRG

- Nu putem demonstra dar credem că există
- D.p.d.v. teoretic putem construi un PRG condiționat bazat pe existența funcțiilor one-way
- În practică construcțiile existente pentru PRG nu pot fi demonstrate ca fiind sigure dar credem că sunt deoarece nu există algoritmi "distinguisher" eficienți



PRG în practică

PRG definite de noi produc tot outputul odata si acesta este de lungime fixa. In practică, PRG-urile sunt instantiate cu sisteme de criptare fluide.

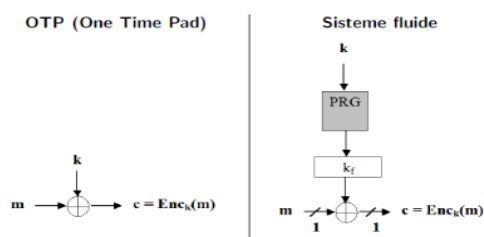
Sisteme fluide

- Sistemele fluide produc bitii de output (pseudo-aleatori) gradual si la cerere, fiind mai eficiente si flexibile. Acestea au 2 faze
 - Se generează o secvență pseudoaleatoare de biți folosind PRG
 - Secvența obținută se XOR-ează cu mesajul clar
- De multe ori cand ne referim la un sistem de criptare fluid considerăm doar faza 1

Sisteme fluide vs OTP

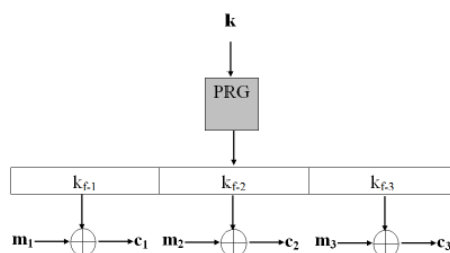
Securitate la interceptare multiplă

- În forma prezentată, sistemul este determinist => nu este sigur
- În practică acesta se folosește în 2 moduri: sincronizat si nesincronizat



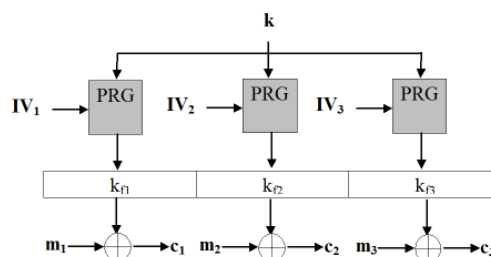
Sincronizat:

- Partenerii folosesc pentru criptare parti succesive ale aceleiasi secvente PR
- Mesajele sunt criptate in mod succesiv
- Necesita pastrarea stării
- Mesajele succesive pot fi percepute ca un singur mesaj clar, lung, obinut prin concatenare
- Se pretează unei singure sesiuni de comunicare



Nesincronizat:

- Mesajele sunt criptate in mod independent
- Nu necesită păstrarea stării
- Valorile IV1, IV2, ..., sunt alese uniform aleator pt fiecare mesaj
- Valorile IV1, IV2 (sau IV la sincronizat) sunt necesare pentru decriptare => fac parte din mesajul criptat

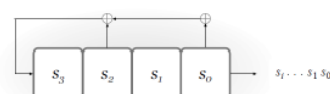


Proprietati ale PRG in modul nesincronizat

Fie $G(s, IV)$ un PRG cu 2 intrări - s = seed și IV = initialization vector. Acesta trebuie să satisfacă:

- $G(s, IV)$ este o secvență pseudoaleatoare chiar dacă IV este public
- Dacă $IV1$ și $IV2$ sunt valori uniform aleatoare atunci $G(s, IV1)$ este indistinctibil de $G(s, IV2)$

4.2 Linear-Feedback Shift Registers (LFSR)



- Se ia bitul s_0 si se xoreaza conform definitiei LFSR-ului si se obtine k . Bitii s_3 ->

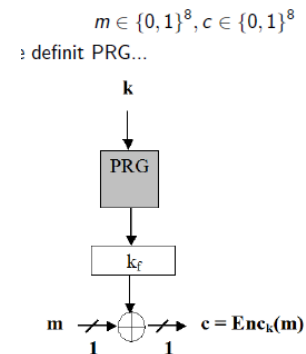
Pentru starea inițială $(0,0,1,1)$, biții de la ieșire vor fi ...
 $(0,0,1,1)$
 $(1,0,0,1)$

s1 se shifteaza la dreapta iar k este pus la stanga (primul bit).

- Starea constă din n biți
- Exista cel mult 2^n stări posibile până se repetă outputul
- Cunoscând cel mult $2n$ biți de ieșire un atacator poate afla starea inițială și coeficienții de feedback
- Vulnerabilități
 - Sunt liniare => vulnerabilitati <= sistemele liniare de ecuatii permit aflarea inf. Sensibile
 - Combinațiile de mai multe LSFR-uri pot produce sisteme de criptare sigure

RC4

- Inventat la MIT de R. Rivest. A devenit public in 1994, utilizat în WEP, SSL/TLS.
- Este un sistem de criptare fluid pe octeți
- 2 faze
 - Initializare: determina starea internă, nu produce chei
 - Generare de chei fluide: modifica starea internă și generează un octet (cheia fluidă) care se XOR-ează cu m pentru a produce c
- Starea internă
 - Un tablou de 256 de octeți
 - 2 indici i și j
- Toate operațiile se efectuează pe octeți (mod 256)
- Detalii de implementare:
 - $5 \leq n \leq 16 \Rightarrow 40 \leq |k| \leq 256$
 - Ocupa 256 de octeți și câteva variabile byte
 - Operații rapide de executat
- Securitate:
 - Primele chei fluide sunt total nealeatorii și oferă informații despre cheie
 - RC4 pe 104 biți a fost spart în aprox 1 min
 - Un atac recent arată că pot fi determinați primii aprox. 200 de octeți din textul clar criptat cu RC4 în TLS cunoscând $[2^{28} - 2^{32}]$ criptări independente



Faza 1. Inițializare

```

▶ n = numărul octeților din cheie,  $1 \leq n \leq 256$ 
▶  $j \leftarrow 0$ 
  for  $i = 0$  to 255 do
     $S[i] \leftarrow i$ 
  end for
  for  $i = 0$  to 255 do
     $j \leftarrow j + S[i] + k[i \pmod n]$ 
    swap ( $S[i], S[j]$ )
  end for
   $i \leftarrow 0$ 
   $j \leftarrow 0$ 

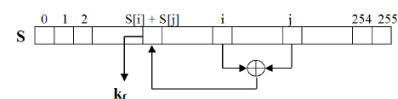
```

Faza 2. Generarea cheii fluide

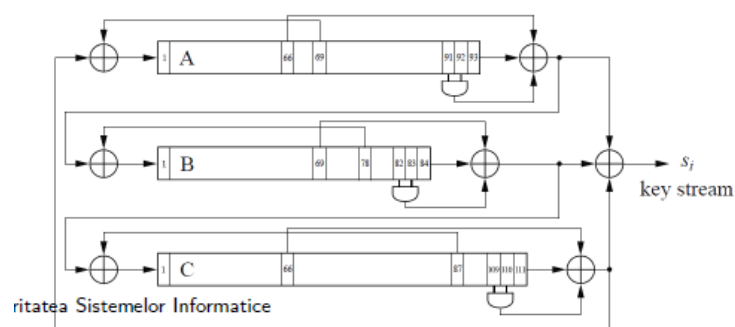
```

▶ cheia se obține octet cu octet
▶  $i \leftarrow i + 1$ 
   $j \leftarrow j + S[i]$ 
  swap ( $S[i], S[j]$ )
  return  $S[S[i] + S[j]]$ 

```



Trivium

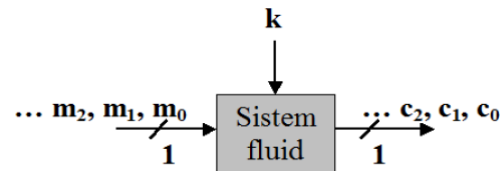
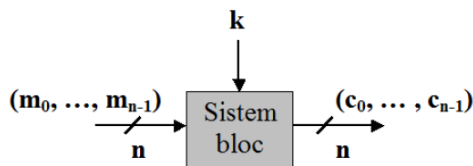


- (2008) simplu și compact hardware - e compus din 3 FSR-uri neliniare de grad 93, 84 și 111

- Regiștri sunt cuplați - la fiecare tact cel mai din stanga registru va contine o valoare calculata ca functie aplicata unui registru din acelasi FSR dar si unor registri din alt FSR
- Cel mai bun atac cunoscut pt Trivium este cel brute-force

Curs 4.3 - Criptografia simetrică, ECB, CBC, OFB, CTR, PRP, PRF

Sisteme de criptare fluide vs bloc



Dpdv. mod de criptare:

- Crijtează câte n biți **simultan**
- Criptarea unui bit din textul clar este **dependentă** de biții din textul clar care aparțin aceluiași bloc

Dpdv. practic:

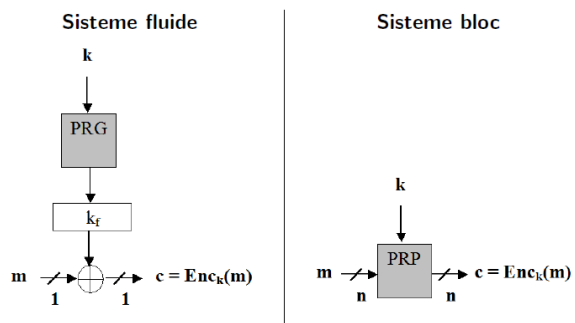
- Necesitati computationale reduse
- Utilizare: mobile, PDA, disp incorpor.
- Par sa fie mai putin sigure

- Crijtează **bit cu bit**
- Criptarea unui bit este independenta de ceilalti

- Necesitati computazionale mai avansate
- Utilizare: internet
- Par sa fie mai sigure

PRP

- Cum **PRG** este necesar in sistemele fluide, **PRP** este necesar in sistemele bloc
- Este o functie **deterministă și bijectivă** care pentru o cheie produce la iesire o **permutare** a intrării **indistinctibilă** de o permutare aleatoare
- Atat functia cat si inversa sunt **usor calculabile**



PRF

- Functia pseudoaleatoare este o generalizare a notiunii de **permutare pseudoaleatoare**
- Aceasta este o functie **cu cheie** care este **indistinctibila** fata de o functie aleatoare (cu acelasi domeniu si multime de valori)
- PRF este generalizare a PRP. PRP este PRF care satisface: $X = Y$, inversabila si calculul inversei este eficient
- Pornind de la PRP se poate construi PRF. **PRP -> PRF este trivial** (vezi bulinuta de deasupra)
- **PRF -> PRG**
 - Fie PRF - $F : K \times \{0, 1\}^n \rightarrow \{0, 1\}^n$;
 - Construim PRG - $G : K \rightarrow \{0, 1\}^{nl}$ PRG $G(k) = F_k(0) || F_k(1) || \dots || F_k(l-1)$
 - Avantaj: constructia este paralelizabila
 - $F_k(.)$ este **indistinctibila** de o functie aleatoare $\Rightarrow G(k)$ este **indistinctibila** de o secventa aleatoare de lungime $nl \Rightarrow G$ este sigur
- **PRF -> PRP Teorema (Luby-Rackoff 5)**
Dacă $F : K \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ este PRF, se poate construi $F' : K \times \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n}$ PRP.

Foloseste runde **Feistel**.

● PRG -> PRF

- Considerăm $G : \mathcal{K} \rightarrow \mathcal{K}^2$ PRG cu $|\mathcal{K}| = n$:

$$G(k) = (G_0(k), G_1(k))$$

$G_0(k)$ și $G_1(k)$ sunt prima și a doua jumătate a ieșirii din PRG

- Construim $F : \mathcal{K} \times \{0, 1\} \rightarrow \mathcal{K}$ PRF:

$$F_k(0) = G_0(k), F_k(1) = G_1(k)$$

$F_k(0)$ întoarce prima jumătate a secvenței pseudoaleatoare $G(k)$

$F_k(1)$ întoarce a doua jumătate a secvenței pseudoaleatoare $G(k)$

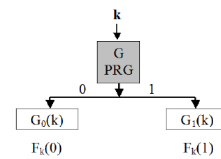
- **Întrebare:** De ce este F sigură?

- **Răspuns:** $G(k)$ este *indistinctibilă* față de o secvență aleatoare de lungime $2n$

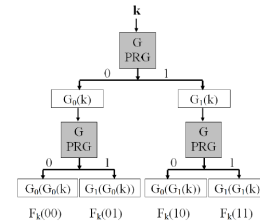
⇒ $G_0(k)$ și $G_1(k)$ sunt *indistinctibile* față de o secvență aleatoare de lungime n

⇒ pentru $k \xleftarrow{R} \{0, 1\}^n$, $F_k(\cdot)$ este *indistinctibilă* față de o funcție aleatoare.

Construcția pentru un singur bit de intrare...



► ...se poate generaliza pentru un număr oarecare de biți



- Poate fi reprez ca un arbore binar cu k radacina
- Valoarea funcției $F_k(x) = F_k(x_0, \dots, x_{n-1})$ este obt. prin parcurgerea arborelui dupa x
- Adancimea este liniara in n
- Dimensiunea este exponentiala in n - 2 la n
- NU se utilizeaza in practica deoarece este ineficient

Moduri de utilizare - ECB/CBC/OFB/CTR

- Dacă lungimea mesajului este **mai mica** decat dimensiunea unui bloc atunci acesta se completeaza cu biti **1 0 ... 0**;
- Dacă lungimea mesajului este **mai mare** atunci se utilizeaza un **mod de operare** (ECB, CBC, OFB, CTR).
Notam cu F_k un sistem de criptare bloc cu k fixat

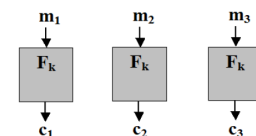


Figure: Imagine preluată de pe <https://en.wikipedia.org/>

Figura din mijloc este criptarea imaginii din stânga în modul ECB. În dreapta este aceeași imagine criptată folosind un mod sigur.

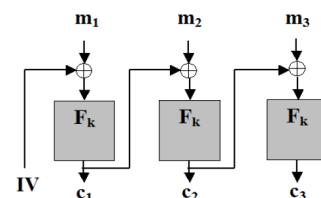
ECB - Electronic code book

- **Paralelizabil, determinist** (nesigur), F_k trebuie sa fie **inversabilă** pentru decriptare
- Un adversar pasiv decteaza repetarea unui bloc pentru ca se repeta blocul criptat coresp.
- **Nu** trebuie folosit



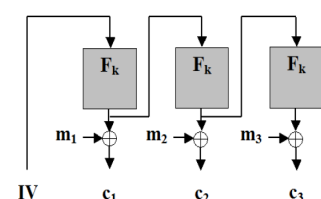
CBC - Cipher block chaining

- IV este ales aleator la criptare
- IV se transmite în clar fiind necesar pt decriptare
- F_k trebuie sa fie **inversabila** pentru decript
- Este **secvential**, un dezavantaj major



OFB - Output FeedBack

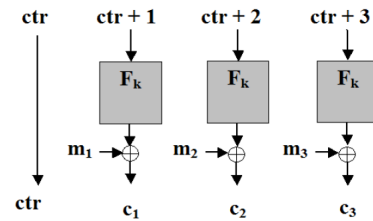
- Genereaza o secventa pseudoaleatoare care se XOR-eaza cu mesajul clar
- IV este ales in mod aleator la criptare
- IV se transmite in clar fiind necesar pt decriptare
- F_k nu trebuie sa fie neaparat inversabila



- Este **secvential** dar secventa pseudoaleatoare poate fi pre-procesata anterior decriptarii

CTR - Counter

- Genereaza o secventa pseudoaleatoare care se XOR-eaza cu mesajul in clar
- Ctr este ales in mod aleator la criptare
- Ctr se transmite in clar
- F_k nu trebuie sa fie neaparat inversabila
- Este **paralelizabil**
- In plus, secventa pseudoaleatoare poate fi pre-procesata anterior decriptarii



Concluzii

- CTR, OFB, CBC sunt CPA-sigure
- CTR, OFB, CBC au un initial value random ceea ce asigura ca f_k este mereu evaluat pe intrari diferite

Curs 5.1- Scheme de criptare CPA-sigure bazate pe PRF

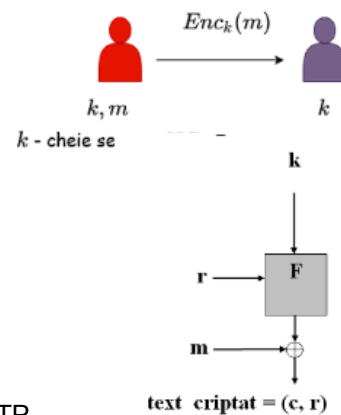
Sisteme de criptare bloc - observații:

1. Sistemele de criptare bloc sunt instantieri sigure ale PRP
Pentru un n suficient de mare, un PRP este indistinctibil de un PRF
2. reamintim ca pentru PRP avem nevoie și de invertibilitate, dar pentru un n suficient de mare, un PRP este și un PRF
3. în practica, sistemele de criptare bloc sunt și PRF bune, nu doar PRP-uri bune

Sistem de criptare CPA-sigur:

- Sistemele de criptare bloc sunt instantieri sigure ale PRP
- Fie F_k o funcție cu cheie
- $\text{Gen}(1^n)$: alege uniform cheie $k \in \{0,1\}^n$
- $\text{Enc}_k(m)$: pentru $|m|=|k|$, alege r uniform in $\{0,1\}^n$

$$\text{Enc}_k(m) = (r, F_k(r) \oplus m)$$
- $\text{Dec}_k(c = (c_0, c_1))$: întoarce $c_1 \oplus F_k(c_0)$



Observații:

- cheia este la fel de lunga precum mesajul - la fel ca la OTP
- dar, spre deosebire de OTP, se pot cripta mai multe mesaje cu aceeași cheie în siguranță

Teorema

Dacă F este PRF, construcția anterioară este o schema de criptare CPA-sigură pentru mesaje de lungime n .

Curs 5.2 - Securitate CCA

Reamintim câteva dintre scenariile de atac pe care le-am mai întâlnit:

- CPA și CCA definite în curs 2

Securitate CCA

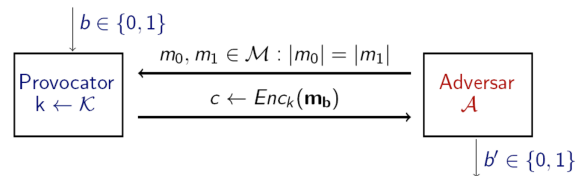
- Capabilitatile adversarului: el poate interactiona cu un oracol de criptare și cu un oracol de decriptare, fiind un adversar activ care poate rula atacuri în timp polinomial
- Adversarul poate transmite către oracolul de criptare orice mesaj și primește înapoi textul criptat corespunzător sau poate transmite către oracolul de decriptare anumite mesaje c și primește înapoi mesajul clar corespunzător
- Dacă sistemul de criptare este nedeterminist, atunci oracolul de criptare folosește de fiecare dată o valoare aleatoare nouă și neutilizată anterior
- Considerăm că securitatea este impactată dacă adversarul poate să distingă între criptările a două mesaje aleatoare

Vom defini securitatea CCA pe baza unui experiment de indistinguibilitate $\text{Priv}_{\mathcal{A},\pi}^{\text{cca}}(n)$ unde $\pi = (\text{Enc}, \text{Dec})$ este schema

- de criptare iar n este parametrul de securitate al schemei π ;
- Personajele participante: adversarul care încearcă să spargă schema și un provocator (challenger);

Experimentul $\text{Priv}_{\mathcal{A},\pi}^{\text{cca}}(n)$

- Pe toată durata experimentului, $\mathcal{A}$ are acces la oracolul de criptare $\text{Enc}(\cdot)$ și la oracolul de decriptare $\text{Dec}(\cdot)$ cu restricția că nu poate decripta c !
- Outputul este 1 dacă $b' = b$ și 0 altfel. Dacă $\text{Priv}_{\mathcal{A}}^{\text{cca}} = 1$ atunci $\mathcal{A}$ a efectuat experimentul cu succes



Definiție

O schemă de criptare $\pi = (\text{Enc}, \text{Dec})$ este **CCA-sigură** dacă pentru orice adversar PPT $\mathcal{A}$ există o funcție neglijabilă negl așa încât

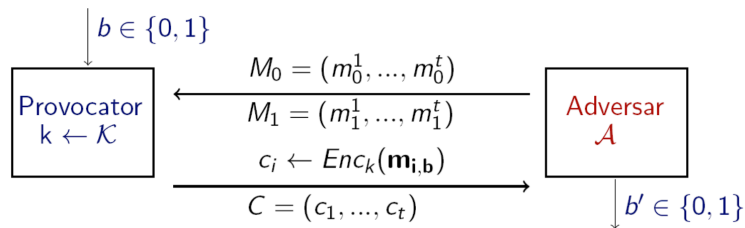
$$\Pr[\text{Priv}_{\mathcal{A},\pi}^{\text{cca}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n).$$

- Un adversar nu poate determina care text clar a fost criptat cu o probabilitate semnificativ mai mare decât dacă ar fi ghicit (în sens aleator, dat cu banul), chiar dacă are acces la oracolele de criptare și decriptare.
- Un sistem **CCA-sigur** este întotdeauna **CPA-sigur**. Experimentul $\text{Priv}_{\mathcal{A}}^{\text{cpa}}(n)$ este $\text{Priv}_{\mathcal{A}}^{\text{cca}}(n)$ în care $\mathcal{A}$ nu folosește oracolul de decriptare.
- Un sistem de criptare determinist **nu poate fi** CCA-sigur, deoarece un sistem determinist nu este CPA-sigur

Securitate CCA - criptare multiplă

În definiția precedentă am considerat cazul unui adversar care primește un singur text criptat. În realitate, în cadrul unei comunicatii se trimit mai multe mesaje pe care adversarul le poate intercepta. Definim ce înseamnă o schemă sigură chiar și în aceste condiții.

Experimentul $\text{Priv}_{\mathcal{A},\pi}^{\text{cca}}(n)$



- Output-ul experimentului este 1 dacă $b'=b$ și 0 altfel
- Definiția de securitate este aceeași, doar ca se referă la experimentul de mai sus.
- Securitatea pentru criptare **simplică** implică securitate pentru criptare **multiplă**!

Concluzie: Nici una din schemele de criptare de până acum nu sunt CCA-sigure.

De reținut:

- Securitate - interceptare simplă $\Rightarrow$ securitate - interceptare multiplă
- Schemele deterministe nu sunt semantic / CPA / CCA sigure
- Securitate CCA $\Rightarrow$ securitate CPA $\Rightarrow$ securitate semantică

Curs 5.3 - Construcții practice PRF, AES, Feistel

Cum se obțin PRF în practică?

Paradigma confuzie-difuzie

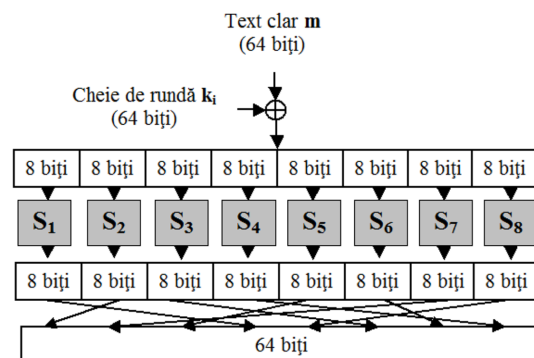
- Se construiește funcția F , pe baza mai multor funcții aleatoare fi de dimensiune mai mică
- Considerăm F pe 128 biți și 16 funcții aleatoare $f_1, \dots, f_{16}$ pe câte 8 biți
Pentru $x = x_1 || \dots || x_{16}$, $x \in \{0, 1\}^{128}$ $x_i \in \{0, 1\}^8$:

- $$F_k(x) = f_1(x_1) || \dots || f_{16}(x_{16})$$
- Spunem că $\{f_i\}$ introduc confuzie în F .

Rețele de substituție - permutare

F încă nu este PRF dar

- F se transformă în PRF în 2 pași:
 - Pasul 1: se introduce difuzie prin amestecarea (permutarea) biților de ieșire;
 - Pasul 2: se repetă o rundă (care presupune confuzie și difuzie) de mai multe ori;
- Repetarea confuziei și difuziei face ca modificarea unui singur bit de intrare să fie propagată asupra tuturor biților de ieșire
- $\{f_i\}$ se numesc S-boxes (Substitution-boxes)
- Cum nu mai depinde de cheie, aceasta este utilizată în alt scop; din cheie se obțin mai multe chei de rundă (sub-chei) în urma unui proces de derivare a cheilor (key schedule)
- Fiecare cheie de rundă este XOR-ata cu valorile intermediare din fiecare rundă



Exista 2 principii de bază în proiectarea rețelelor de substituție-permutare:

- Principiul 1: Inversabilitatea S-box-urilor
 - dacă toate S-box-urile sunt inversabile, atunci rețeaua este inversabilă
 - necesitate funcțională (pentru decriptare)
- Principiul 2: Efectul de avalanșă
 - Un singur bit modificat la intrare trebuie să afecteze toți biții din secvența de ieșire
 - necesitate de securitate

AES

Descriere AES

- AES este o rețea de substituție - permutare pe 128 biți care poate folosi chei de 128, 192 sau 256 biți

Lungimea cheii determină numărul de runde:

Lungime cheie (biți)	128	192	256
Număr runde	10	12	14

- Folosește o matrice de octeți 4×4 numită stare
- Starea inițială este mesajul clar (4×4×8 = 128)
- Ieșirea din ultima rundă este textul criptat

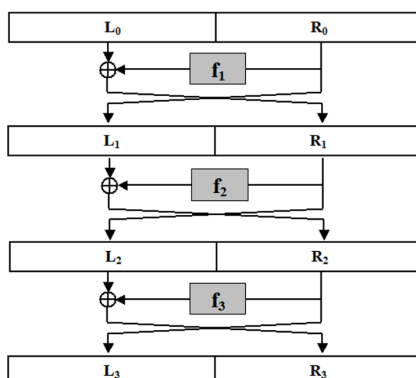
Securitatea sistemului AES

- Singurele atacuri netriviiale sunt asupra AES cu număr redus de runde
 - AES-128 cu 6 runde: necesită 2^{72} criptări
 - AES-192 cu 8 runde: necesită 2^{188} criptări
 - AES-256 cu 8 runde: necesită 2^{204} criptări
- Nu există un atac mai eficient decât căutarea exhaustivă pentru AES cu număr complet de runde

Rețele Feistel

Descriere

- Se aseamănă rețelelor de substituție-permutare
- Se diferențiază de rețelele de substituție-permutare prin proiectarea de nivel înalt
- Introduc avantajul major că S-box-urile NU trebuie să fie inversabile
- Permit așadar obținerea unei structuri inversabile folosind elemente neinvertabile



Intrarea în runda i se împarte în 2 jumătăți: L_{i-1} și R_{i-1} (i.e. *Left* și *Right*);

Ieșirile din runda i sunt:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus f_i(R_{i-1})$$

Funcțiile f_i depind de cheia de rundă, derivând dintr-o funcție publică $\hat{f}_i$:

$$f_i(R) = \hat{f}_i(k_i, R)$$

Rețelele Feistel sunt inversabile indiferent dacă funcțiile f_i sunt inversabile sau nu;

Fie (L_i, R_i) ieșirile din runda i ;

Intrările (L_{i-1}, R_{i-1}) în runda i sunt:

$$R_{i-1} = L_i$$

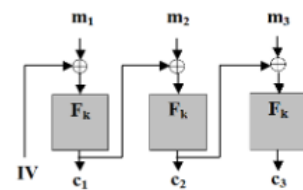
$$L_{i-1} = R_i \oplus f_i(R_{i-1})$$

Curs 6.1 - Padding-oracle attack

Atacul bazat pe oracol de padding

- Am văzut cum funcționează modul CBC când lungimea mesajului clar este multiplu de lungimea L blocului de criptat (suportat de F_k) în octeți
Ce se întâmplă când $|m| \neq L$?
- Folosim padding-ul PKCS#7 :
 - Fie b numărul de octeți de adăugat la ultimul bloc pentru a avea $|m|$ multiplu de L ($1 \leq b \leq L$)
 - Se adaugă b octeți la ultimul bloc din m , fiecare reprezentând valoarea lui b

Diagrama 6.1.1: Modul de criptare CBC



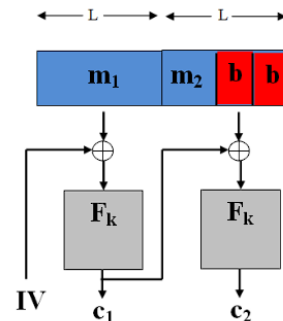
Decriptare

$$m_1 = F_k^{-1}(c_1) \oplus IV$$

$$m_2 = F_k^{-1}(c_2) \oplus c_1$$

$$m_3 = F_k^{-1}(c_3) \oplus c_2$$

Criptare



CBC cu padding PKCS7

Considerăm situația în care un client trimite mesaje criptate în modul CBC către un server.

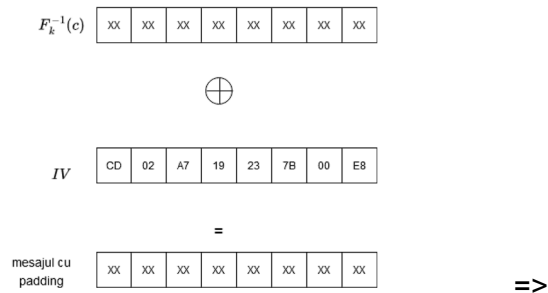
- În urma decriptării se obțin $m_1 || m_2$
- se citește octetul final din b
- dacă ultimii b octeți au toți valoarea b , atunci se scoate padding-ul și se obține mesajul original m
- altfel întoarce mesajul padding greșit și cerere transmiterea mesajului

Server-ul acționează ca un oracol de padding - adversarul îi trimite texte criptate și afla dacă padding-ul este corect sau nu.

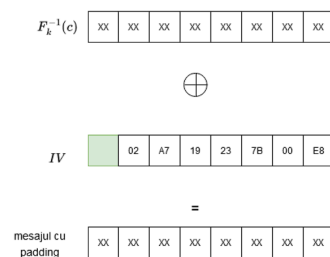
Ideea atacului cu oracol de padding

- Unui text criptat (IV, c) îi corespunde textul clar cu padding

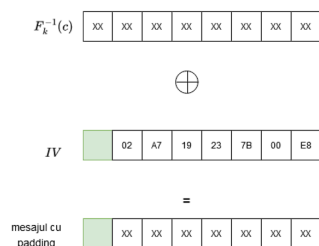
$$m' = F_k^{-1}(c) \oplus IV$$
- Dacă un adversar modifică octetul i din IV atunci modificarea se va reflecta și în octetul i din m'



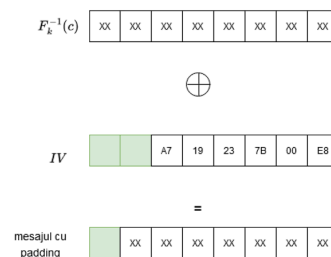
Adversarul modifica primul octet din IV



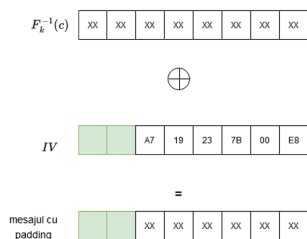
Modificarea se reflectă în primul octet din mesajul cu padding; apoi trimite mesajul (IV', c) și verifică dacă primește eroare



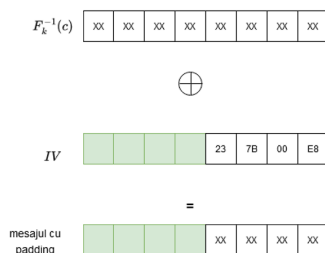
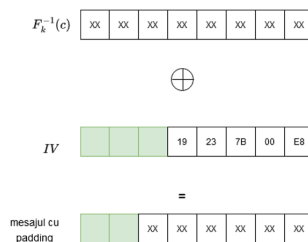
In caz contrar, adversarul modifică al 2-lea octet din IV



Modificarea se reflectă în al 2-lea octet din mesajul cu padding; apoi trimite mesajul (IV', c) și verifică dacă primește eroare



In caz contrar, adversarul continua cu al 3-lea octet din IV



- Eroare la decriptare
- adversarul deduce ca oracolul verifica ultimii 5 octeti, care au valoarea 05

... se ajunge la

$F_k^{-1}(c)$

xx	xx	xx	xx	xx	xx	xx	xx	xx
----	----	----	----	----	----	----	----	----

$\oplus$

IV

CD	02	A7	19	23	7B	00	E8
----	----	----	----	----	----	----	----

=

mesajul cu padding

xx	xx	xx	05	05	05	05	05
----	----	----	----	----	----	----	----

Primii 3 octeti din mesajul cu padding sunt încă necunoscuți atacatorului

=>

Adversarul încearcă să găsească primii 3 octeți din mesajul cu padding
Modificarea se va reflecta în cel mai din dreapta octet din mesajul cu padding

$F_k^{-1}(c)$

xx	xx	xx	xx	xx	xx	xx	xx
----	----	----	----	----	----	----	----

$\oplus$

IV

CD	02	A7	19	23	7B	00	EB
----	----	----	----	----	----	----	----

modifică pentru a obține
 $E8 \oplus 05 \oplus 06$

=

mesajul cu padding

xx	xx	xx	05	05	05	05	06
----	----	----	----	----	----	----	----

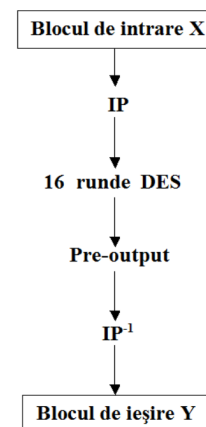
Complexitatea atacului cu oracol de padding

- sunt necesare cel mult L încercări pentru a afla b
- cel mult $2^8 = 256$ încercări pentru a afla fiecare octet din mesajul inițial
- în total sunt necesare $256 * bt$ încercări (unde bt reprezintă numărul de octeți din mesajul original) pentru a găsi întregul text clar

Curs 6.3 - Data Encryption Standard (DES)

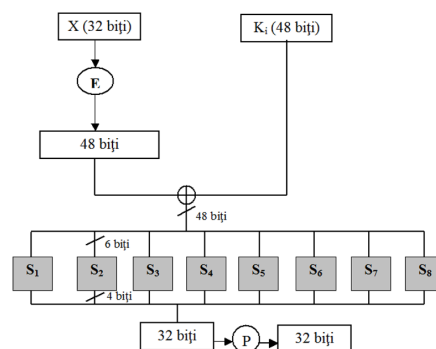
Descriere DES

- DES este o rețea de tip Feistel cu 16 runde și o cheie pe 56 biți
- Procesul de derivare a cheii (key schedule) obține o sub-cheie de runda k_i pentru fiecare runda pornind de la cheia master k
Funcțiile de rundă $f_i(R) = f(k_i, R)$ sunt derivate din aceeași funcție principală $\hat{f}$ și nu sunt inversabile;
- Fiecare sub-cheie k_i reprezintă permutarea a 48 biți din cheia master
- Întreaga procedură de obținere a sub-cheilor de runda este fixă și cunoscută, singurul secret este cheia master



Funcția $f(k_i, R)$

- Funcția f este, în esență, o rețea de substituție-permutare cu doar o rundă
- Pentru calculul $f(k_i, R)$ se procedează astfel:
 - a. R este expandat la un bloc R' de 48 biți cu ajutorul unei funcții de expandare $R' = E(R)$
 - b. R' este XOR-at cu k_i iar valoarea rezultată este împărțită în 8 blocuri de câte 6 biți
 - c. Fiecare bloc de 6 biți trece printr-un SBOX diferit rezultând o valoare pe 4 biți
 - d. Se concatenează blocurile rezultate și se aplică o permutare, rezultând în final un bloc de 32 biți
- **De remarcat:** Toată descrierea lui DES, inclusiv SBOX-urile și permutările sunt publice



SBOX-urile din DES

- Formează o parte esențială din construcția DES

- DES devine mult mai vulnerabil la atacuri dacă SBOX-urile sunt modificate ușor sau dacă sunt alese aleator
- Primul și ultimul bit din cei 6 de la intrare sunt folosiți pentru a alege linia din tabel, iar biții 2-5 sunt folosiți pentru coloana; output-ul va consta din cei 4 biți aflați la intersecția liniei și coloanei alese

S ₅		Cei 4 biți din mijloc															
		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Biții din margine	00	0010	1100	0100	0001	0111	1010	1011	0110	1000	0101	0011	1111	1101	0000	1110	1001
	01	1110	1011	0010	1100	0100	0111	1101	0001	0101	0000	1111	1010	0011	1001	1000	0110
	10	0100	0010	0001	1011	1010	1101	0111	1000	1111	1001	1100	0101	0110	0011	0000	1110
	11	1011	1000	1100	0111	0001	1110	0010	1101	0110	1111	0000	1001	1010	0100	0101	0011

- Modificarea unui bit de la intrare întotdeauna afectează cel puțin doi biți de la ieșire
- DES are un puternic efect de avalanșă generat de ultima proprietate menționată mai sus și de permutările folosite

Securitatea sistemului DES

- Încă de la propunerea sa, DES a fost criticat din doua motive
 - a. Spațiul cheilor este prea mic facand algoritmul vulnerabil la forta bruta
 - b. Criteriile de selecție a SBOX-urilor au fost ținute secrete și ar fi putut exista atacuri analitice care explorau proprietățile matematice ale SBOX-urilor, cunoscute numai celor care l-au proiectat
- O alta problema a sistemului DES, mai puțin importantă, este lungimea blocului relativ scurta (64 biți)

Criptanaliză avansată

1. Criptanaliza diferențială
 - a. Atacul presupune complexitate timp 2^{37} (memorie neglijabilă) dar cere ca atacatorul sa analizeze 2^{36} texte criptate obținute dintr-o mulțime de 2^{47} texte clare alese
 - b. Din punct de vedere teoretic, atacul a fost o inovație, dar practic e aproape imposibil de realizat
2. Criptanaliza liniară
 - a. Deși necesita 2^{43} texte criptate, avantajul este că textele clare nu trebuie sa fie alese de atacator, ci doar cunoscute de el
 - b. Problema însă ramane aceeași: atacul e foarte greu de pus în practică

Criptanaliza diferențială

- Fie un cifru bloc cu blocul de lungime n și $\Delta_x, \Delta_y \in \{0, 1\}^n$;
- Spunem că **diferențiala** (Δ_x, Δ_y) apare cu probabilitate p dacă pentru intrări aleatoare x_1, x_2 cu

$$x_1 \oplus x_2 = \Delta_x$$

și o alegere aleatoare a cheii k

$$Pr[F_k(x_1) \oplus F_k(x_2) = \Delta_y] = p$$

- Pentru o funcție aleatoare, probabilitatea de apariție a unei diferențiale nu e mai mare decât 2^{-n} ;
- La un cifru bloc slab, ea apare cu o probabilitate mult mai mare;
 - Dacă o diferențiala exista cu probabilitate $p \gg 2^{-(n)}$, cifrul bloc nu mai este permutare pseudoaleatoare

- Ideea este de a folosi multe diferențiale cu p ușor mai mare decât $2^{-(n)}$ pentru a găsi cheia secretă folosind un atac cu text clar ales
- Criptanaliza diferențiată a fost folosită cu succes pentru a ataca cifruri bloc (altele decât DES și AES), de pildă FEAL-8

Criptanaliza liniară

- Metoda consideră relații liniare între intrările și ieșirile unui cifru bloc
Spunem că porțiunile de biți $i_1, \dots, i_l$ și $i'_1, \dots, i'_l$ au distanța p dacă pentru orice intrare aleatoare x și orice cheie k

$$\Pr[x_{i_1} \oplus \dots \oplus x_{i_l} \oplus y_{i'_1} \oplus \dots \oplus y_{i'_l} = 0] = p$$

- unde $y = F_k(x)$.
- Pentru o funcție aleatoare se așteaptă ca $p = 0.5$

Creșterea lungimii cheii

- Singura vulnerabilitate practică DES este cheia scurtă
- Nu se recomandă schimbarea structurii interne întrucât securitatea sistemului ar putea fi afectată
- Soluție alternativă: considerăm DES o cutie neagră care implementează un cifru bloc "perfect" cu o cheie pe 56 biți

Criptare dubla

Fie F un cifru bloc (în particular ne vom referi la DES);
definim un alt cifru bloc F' astfel

$$F'_{k_1, k_2}(x) = F_{k_2}(F_{k_1}(x))$$

- cu k_1, k_2 chei independente;
- Lungimea totală a cheii este 112 biți, suficient de mare pentru cautare exhaustivă
- Însă, se poate arăta un atac în timp 2^{56} unde $|k_1| = 56 = |k_2|$ (față de 2^{112} cat necesită o cautare exhaustivă)
- Atacul se numește **meet-in-the-middle**

Atacul meet-in-the-middle

- Iată cum funcționează atacul dacă se cunoaște o pereche text clar/text criptat (x, y) cu $y = F_{k_2}(F_{k_1}(x))$:

1. Pentru fiecare $k_1 \in \{0, 1\}^n$, calculează $z := F_{k_1}(x)$ și păstrează (z, k_1) ;
2. Pentru fiecare $k_2 \in \{0, 1\}^n$, calculează $z := F_{k_2}^{-1}(y)$ și păstrează (z, k_2) ;
3. Verifică dacă există perechi (z, k_1) și (z, k_2) care coincid pe prima componentă;
4. Atunci valorile k_1, k_2 corespunzătoare satisfac

$$F_{k_1}(x) = F_{k_2}^{-1}(y)$$

adică $y = F'_{k_1, k_2}(x)$

- Complexitatea timp a atacului este $O(2^n)$.

Criptare triplă

Există două variante:

1. Trei chei independente - k_1, k_2 și k_3 iar

$$F'_{k_1, k_2, k_3} = F_{k_3}(F_{k_2}^{-1}(F_{k_1}(x)))$$

2. Două chei independente - k_1 și k_2 iar

$$F'_{k_1, k_2} = F_{k_1}(F_{k_2}^{-1}(F_{k_1}(x)))$$

-
- F' este ales astfel pentru a fi compatibil cu F atunci când cheile sunt alese $k_1=k_2=k_3$
- Prima variantă are lungimea cheii $3n$ dar cel mai bun atac necesită timp 2^{2n} (funcționează atacul meet-in-the-middle)
- A doua variantă are lungimea cheii $2n$ și cel mai bun atac necesită timp 2^{2n}

Triplu-DES (3DES)

- Se bazează pe triplă invocare a lui DES folosind două sau trei chei
- 3DES este foarte eficient în implementările hardware (la fel ca și DES) dar totuși lent în implementări software
- Singurele dezavantaje ar fi lungimea mică a blocurilor și faptul că este destul de lent fiindcă aplică DES de trei ori
- Acestea au dus la înlocuirea lui ca standard cu AES
- DES-X este o variantă DES care rezistă mai bine (decat DES) la forța brută
DES-X folosește două chei suplimentare k_1, k_2 :

$$DESX_{k, k_1, k_2} = k_2 \oplus DES_k(x \oplus k_1)$$

•

De reținut:

- DES cu cheia pe 56 biți poate fi spart relativ ușor astăzi prin forța brută
- Însă, este foarte greu de spart folosind criptanaliza diferențială sau liniară
- Pentru 3DES nu se cunoaște nici un atac practic

Curs 7.1 - MAC

Criptare vs autentificarea mesajelor

- Criptarea, în general, NU oferă integritatea mesajelor!
- Nu folosiți criptarea cu scopul de a obține autentificarea mesajelor
- Dacă un mesaj este transmis criptat de-a lungul unui canal de comunicare, nu înseamnă că un adversar nu poate modifica/altera mesajul așa încât modificarea să aibă sens în textul clar.

Nici o schema de criptare studiată nu oferă integritatea mesajelor:

- Criptarea folosind sisteme fluide:
 - Dacă modificăm un singur bit din textul criptat c , modificarea se va reflecta imediat acelasi bit din textul clar
 - Acelasi atac se poate aplica și la OTP
- Criptarea folosind sisteme bloc:
 - Atacul se aplică identic pt OFB și CTR.
 - Pentru modul ECB, modificarea unui bit din al i -lea bloc criptat afectează numai al i -lea bloc clar, dar este foarte greu de prezis efectul exact;
 - Mai mult, ordinea blocurilor la ECB poate fi schimbată;
 - Pentru modul CBC, schimbarea bitului j din IV va schimba bitul j din primul bloc

- Toate celelalte blocuri de text clar rămân neschimbate ($m_i = F_k^{-1}(c_i) \oplus c_{i-1}$ iar blocurile c_i și c_{i-1} nu au fost modificate).

Coduri de autentificare a mesajelor - MAC

Pentru a rezolva problema autentificării mesajelor vom folosi un mecanism diferit, numit cod de autentificare a mesajelor - MAC; Scopul lor este de a împiedica un adversar să modifice un mesaj trimis fără ca partile care comunica să nu detecteze modificarea;

Vom lucra în continuare în contextul criptografiei cu cheie secretă unde partile trebuie să stabilească de comun acord o cheie secretă.

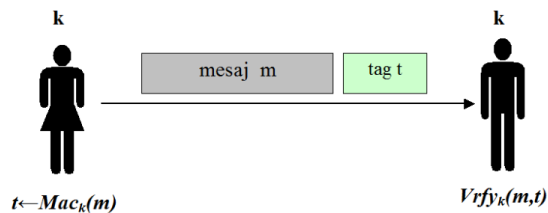
Definiție

Alice și Bob stabilesc o cheie secretă k pe care o partajează;

Când Alice vrea să îi trimită un mesaj m lui Bob, calculează mai întâi un tag t (o etichetă) pe baza mesajului m și a cheii k și trimite perechea (m, t) ;

Tag-ul este calculat folosind un algoritm de generare a tag-urilor numit Mac ;

La primirea perechii (m, t) Bob verifică dacă tag-ul este valid (în raport cu cheia k) folosind un algoritm de verificare Vrfy ;



Un cod de autentificare a mesajelor (MAC) definit peste $(\mathcal{K}, \mathcal{M}, \mathcal{T})$ este format dintr-un triplet de algoritmi polinomiali $(\text{Gen}, \text{Mac}, \text{Vrfy})$ unde:

1. $\text{Gen}(1^n)$: este algoritmul de generare a cheii k (aleasă uniform pe n biți)
2. $\text{Mac} : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{T}$ este algoritmul de generare a tag-urilor $t \leftarrow \text{Mac}_k(m)$;
3. $\text{Vrfy} : \mathcal{K} \times \mathcal{M} \times \mathcal{T} \rightarrow \{0, 1\}$ este algoritmul de verificare ce întoarce un bit $b = \text{Vrfy}_k(m, t)$ cu semnificația că:
 - $b = 1$ înseamnă valid
 - $b = 0$ înseamnă invalid

$$a.\hat{t} : \forall m \in \mathcal{M}, k \in \mathcal{K} \text{ Vrfy}_k(m, \text{Mac}_k(m)) = 1.$$

Cazuri de folosire ale MAC

- Când cele 2 parti care comunica partajează o cheie secretă în avans
- Când avem o singură parte care autentifică o comunicare interacționând cu ea însăși după un timp

Securitate MAC

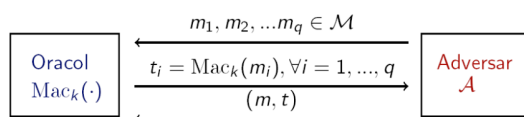
Intuiție: nici un adversar polinomial nu ar trebui să poată genera un tag valid pentru nici un mesaj "nou" care nu a fost deja trimis (și autentificat) de partile care comunica.

Formalizare: Îl dăm adversarului acces la un oracol $\text{Mac}_k(\cdot)$. Adversarul poate trimite orice mesaj m către oracol și primește înapoi un tag corespunzător $\text{tag} \leftarrow \text{Mac}_k(m)$.

Securitatea este impactată dacă adversarul este capabil să producă un mesaj m împreună cu un tag t așa încât:

1. t este un tag valid pt mesajul m : $\text{Vrfy}_k(m, t) = 1$
2. Adversarul nu a solicitat anterior de la oracol un tag pentru m

Experimentul $\text{Mac}_{\mathcal{A}, \pi}^{\text{forge}}(n)$



- Output-ul experimentului este 1 dacă și numai dacă:
 - (1) $\text{Vrfy}_k(m, t) = 1$ și (2) $m \notin \{m_1, \dots, m_q\}$;
- Dacă $\text{Mac}_{\mathcal{A}, \pi}^{\text{forge}}(n) = 1$, spunem că $\mathcal{A}$ a efectuat experimentul cu succes.

Intrebare: De ce este necesara a doua conditie de la securitatea MAC (un adversar nu poate intoarce un mesaj pentru care anterior a cerut un tag)?

Raspuns: Pentru a evita atacurile triviale in care un adversar cere tag-ul aferent unui mesaj si apoi intoarce chiar acel mesaj impreuna cu tag-ul primit.

Intrebare: Definitia MAC oferaa protectie la atacurile prin replicare (in care un adversar copiaza un mesaj impreuna cu tag-ul aferent trimise de partile comunicante)?

Raspuns: Nu! Mac-urile nu ofera protectie la atacurile prin replicare pentru ca definitia nu incorporeaza nici o notiune de stare in algoritmul de verificare

Definiție

Un cod de autentificare al mesajelor $\pi = (\text{Gen}, \text{Mac}, \text{Vrfy})$ este sigur (nu poate fi falsificat printr-un atac cu mesaj ales) dacă pentru orice adversar polinomial $\mathcal{A}$ există o funcție neglijabilă negl așa încât

$$\Pr[\text{Mac}_{\mathcal{A}, \pi}^{\text{forge}}(n) = 1] \leq \text{negl}(n).$$

Constructia MAC-urilor sigure

Funcțiile pseudoaleatoare (PRF) sunt un instrument bun pentru a construi MAC-uri sigure
Constructia urmatoare functioneaza doar pe mesaje de lungime fixa

Construcție

Fie $F : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ o PRF. Definim un MAC în felul următor:

- **Mac** : pentru o cheie $k \in \{0, 1\}^n$ și un mesaj $m \in \{0, 1\}^n$, calculează tag-ul $t = F_k(m)$ (dacă $|m| \neq |k|$ nu întoarce nimic);
- **Vrfy** : pentru o cheie $k \in \{0, 1\}^n$, un mesaj $m \in \{0, 1\}^n$ și un tag $t \in \{0, 1\}^n$, întoarce 1 dacă și numai dacă $t = F_k(m)$ (dacă $|m| \neq |k|$, întoarce 0).

Teoremă

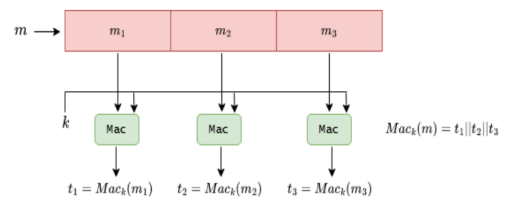
Dacă F este PRF, construcția de mai sus reprezintă un cod de autentificare a mesajelor sigur (nu poate fi falsificat prin atacuri cu mesaj ales).

MAC-uri pentru mesaje de lungime variabila

Pornim de la un MAC de lungime fixa. Fie $(\pi' = (\text{Mac}', \text{Vrfy}'))$ un MAC sigur de lungime fixa pentru mesaje de lungime n . Pentru a construi un MAC de lungime variabila, putem sparge mesajul m în blocuri $m_1, \dots, m_d$ și autentificăm blocurile folosind π' .

Iata cateva modalitati de a face asta:

2. Autentificare separată pentru fiecare bloc:



► **Intrebare:** Este sigură această metodă?

► **Răspuns:** NU!

► Fiind dat (m, t) cu $m = m_1 || m_2 || m_3$ și $t = t_1 || t_2 || t_3$

► Atunci (m', t') este o pereche validă cu $m' = m_2 || m_1 || m_3$ și $t' = t_2 || t_1 || t_3$

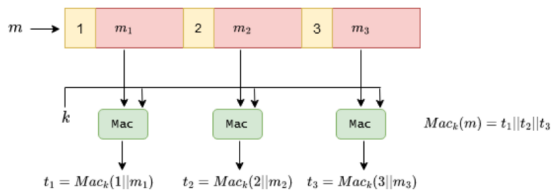
1. XOR pe toate blocurile cu autentificarea rezultatului:

$$t = \text{Mac}'_k(\oplus_i m_i)$$

- **Intrebare:** Este sigură această metodă?
- **Răspuns:** NU! Un adversar poate modifica mesajul original m a.î. XOR-ul blocurilor nu se schimbă, el obținând un tag valid pentru un mesaj nou;

3. Autentificare separată pentru fiecare bloc folosind o secvență de numere:

În felul acesta prevenim atacul anterior

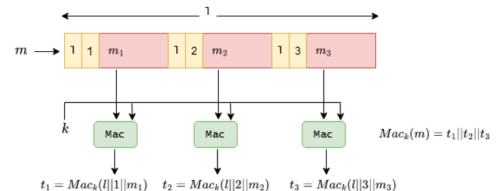


- **Intrebare:** Este sigură această metodă?
- **Răspuns:** NU! Un adversar poate scoate blocuri de la sfârșitul mesajului: $t = t_1||t_2$ este un tag valid pentru mesajul $m = m_1||m_2$;

CBC-MAC

O soluție mult mai eficientă pt MAC-uri sigure este să folosim CBC-MAC. Aceasta

- Soluție pentru atacurile anterioare: adăugarea de informație suplimentară în fiecare bloc



- **Intrebare:** Este sigură această metodă?
- **Răspuns:** NU!
- Pentru perechea (m, t) cu $m = m_1||m_2||m_3$ și $t = t_1||t_2||t_3$ și perechea (m', t') cu $m' = m'_1||m'_2||m'_3$ și $t' = t'_1||t'_2||t'_3$ iar $|m| = |m'|$

este o construcție similară cu modul CBC folosit pentru criptare.

Folosind CBC-MAC pentru un tag aferent

unui mesaj de lungime $l \cdot n$ se aplica sistemul bloc doar de l ori, iar tag-ul are numai n biți

Dacă F este o funcție pseudoaleatoare, construcția de mai sus reprezintă un cod de autentificare a mesajelor sigur (nu poate fi falsificat prin atacuri cu mesaj ales) pentru mesaje de lungime $l \cdot n$ pentru l fixat. Construcția prezentată este sigură numai pentru autentificarea mesajelor de lungime fixă; l fixat înseamnă că cele două părți care comunică trebuie să partajeze l în avans. Avantajul acestei construcții față de cea anterioară este că ea poate autentifica mesaje de lungime mult mai mare;

Construcții MAC: Există 2 construcții de bază ce se folosesc în practică: CBC-MAC (industria bancară) și HMAC (pt protocoale pe internet - SSL, IPsec, SSH,...)

HMAC

$$\text{Mac}(k, m): t = H((k \oplus \text{opad}) || H((k \oplus \text{ipad}) || m));$$

$$\text{Vrfy}(k, m, t) = 1 \iff t = \text{Mac}(k, m).$$

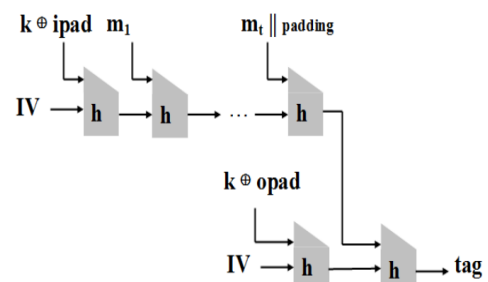
Notatii:

ipad și opad sunt două constante de lungimea unui bloc m_i ipad constă din byte-ul 0x5C repetat de atâtea ori cât e nevoie; opad constă din byte-ul 0x36 repetat de atâtea ori cât e nevoie; IV este o constantă fixată.

Confidențialitate și integritate - criptare autentificată:

Confidențialitate obținem din scheme de criptare, integritate garantăm cu MAC-uri, în practică avem nevoie de ambele proprietăți de securitate, adică de **criptare autentificată**. Nu orice combinație de schema de criptare sigură și MAC sigur oferă cele 2 proprietăți de securitate!!

Iată 3 abordări uzuale pentru a combina criptarea și autentificarea mesajelor:



1. **Criptare-și-autentificare:** criptarea și autentificarea se fac independent. Pentru un mesaj clar m , se transmite mesajul criptat $\langle c, t \rangle$ unde

$$c \leftarrow \text{Enc}_{k_1}(m) \text{ și } t \leftarrow \text{Mac}_{k_2}(m)$$
 La recepție, $m = \text{Dec}_{k_1}(c)$ și dacă $\text{Verf}_{k_2}(m, t) = 1$, atunci întoarce m ; altfel întoarce $\perp$.

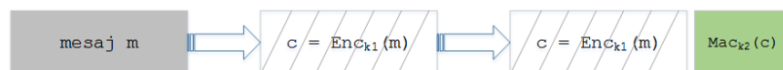
2. **Autentificare-apoi-criptare:** întâi se calculează tag-ul t apoi mesajul și tag-ul sunt criptate împreună

$$t \leftarrow \text{Mac}_{k_2}(m) \text{ și } c \leftarrow \text{Enc}_{k_1}(m||t)$$
 La recepție, $m||t = \text{Dec}_{k_1}(c)$ și dacă $\text{Verf}_{k_2}(m, t) = 1$, atunci întoarce m ; altfel întoarce $\perp$.



- Combinația aceasta **nu** este neapărat sigură; un MAC sigur implică nici un fel de confidențialitate;
- Se poate construi o schemă de criptare CPA-sigură care împreună cu orice MAC sigur nu poate fi CCA-sigură;

3. Criptare-apoi-autentificare



- Combinația aceasta este întotdeauna sigură; se folosește în IPsec;
- Deși folosim aceeași construcție pentru a obține securitate CCA și transmitere sigură a mesajelor, scopurile urmărite sunt diferite în fiecare caz;

Necesitatea de a folosi chei diferite:

Sa urmarim ce se intampla daca folosim metoda 3 cu aceeasi cheie k atat pentru criptare cat si pentru autentificare:

Definim $\text{Enc}_k(m) = F_k(m||r)$, pentru $m \in \{0, 1\}^{n/2}$, $r \leftarrow \{0, 1\}^{n/2}$ aleator, iar $F_k(\cdot)$ o permutare pseudoaleatoare;

Definim $\text{Mac}_k(c) = F_k^{-1}(c)$;

Schema de criptare și MAC-ul sunt sigure dar...

$$\text{Enc}_k(m), \text{Mac}_k(\text{Enc}_k(m)) = F_k(m||r), F_k^{-1}(F_k(m||r)) = F_k(m||r), m||r.$$

Securitate CCA vs Criptare autentificata

Orice schema de criptare autentificata este si CCA-sigura dar exista si scheme de criptare CCA-sigure care nu sunt scheme de criptare autentificata. Totusi, cele mai multe aplicatii de scheme de criptare cu cheie secretă în prezența unui adversar activ necesita integritate

Criptare autentificata in practica

CA in	bazata pe	care in general este	dar in acest caz este
SSH	C_si_A	nesigura	sigura
SSL	A_apoi_C	nesigura	nesigura
IPSec	C_apoi_A	sigura	sigura
WinZip	C_apoi_A	sigura	nesigura

7.2 - Functii Hash

Daca in contextul structurilor de date este preferabil sa se minimizeze coliziunile, in criptografie acest lucru este impus. Daca in contextul structurilor de date coliziunile apar arbitrar, in criptografie coliziunile trebuie evitate daca chiar sunt cautate in mod voit(de catre adversar).

Intrebare: Exista functii hash fara coliziuni?

Raspuns: Nu! Functiile hash comprima, deci functia nu poate fi injectiva.

Considerăm funcțiile hash de domeniu infinit și ieșire de lungime fixată $l(n)$, unde $l(n)$ este un polinom în parametrul de securitate n : $H: \{0, 1\}^* \rightarrow \{0, 1\}^{l(n)}$.

Experiment:

- ▶ Considerăm experimentul $\text{Hash}_{\mathcal{A}, H}^{\text{coll}}(n)$;
- ▶ Adversarul $\mathcal{A}$ indică 2 valori $x, x' \in \{0, 1\}^*$;
- ▶ Se definește valoarea experimentului $\text{Hash}_{\mathcal{A}, H}^{\text{coll}}(n) = 1$ dacă $x \neq x'$ și $H(x) = H(x')$;
- ▶ Altfel, $\text{Hash}_{\mathcal{A}, H}^{\text{coll}}(n) = 0$.

Rezistența la coliziuni:

Definiție

$H: \{0, 1\}^* \rightarrow \{0, 1\}^{l(n)}$ se numește **rezistentă la coliziuni** (*collision-resistant*) dacă pentru orice adversar PPT $\mathcal{A}$ există o funcție neglijabilă negl așa încât

$$\Pr[\text{Hash}_{\mathcal{A}, H}^{\text{coll}}(n) = 1] \leq \text{negl}(n).$$

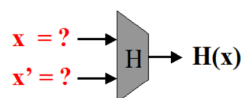
Securitatea funcțiilor

hash:

În practică, rezistența la coliziuni poate fi dificil de obținut. Pentru anumite aplicații sunt utile noțiuni mai relaxate de securitate. Există 3 nivele de securitate:

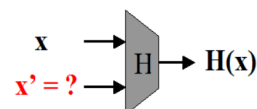
- Rezistența la coliziuni: Este cea mai puternică noțiune de securitate
- Rezistența la a doua preimagine: Presupune că fiind dat x este dificil de determinat $x' \neq x$ a.i. $H(x) = H(x')$
- Rezistența la prima preimagine: presupune că fiind dat $H(x)$ este imposibil de determinat x .

Rezistența la coliziuni



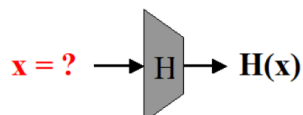
- ▶ **Provocare:** Se cer 2 valori $x \neq x'$ a.i. $H(x) = H(x')$;
- ▶ **Atac:** "Atacul zilei de naștere" necesită $\approx 2^{l(n)/2}$ evaluări pentru H .

Rezistența la a doua preimagine



- ▶ **Provocare:** Fiind dat x , se cere $x' \neq x$ a.i. $H(x) = H(x')$;
- ▶ **Atac:** Un atac generic necesită $\approx 2^{l(n)}$ evaluări pentru H .

Rezistența la prima preimagine:



- ▶ **Provocare:** Fiind dat $y = H(x)$, se cere x a.i. $H(x) = y$;
- ▶ O astfel de funcție se numește și *calculabilă într-un singur sens* (*one-way function*);

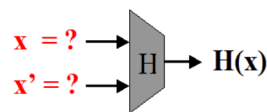
Intrebare: De ce o functie care satisface proprietatea de rezistenta la coliziuni satisface si proprietatea de rezistenta la a doua preimagine?

Raspuns: Pentru x fixat, adversarul determina $x' \neq x$ pentru care $H(x) = H(x')$, deci gaseste o coliziune

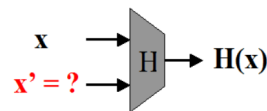
Intrebare: De ce o functie care satisface proprietatea de rezistenta la a doua preimagine satisface si proprietatea de rezistenta la prima imagine?

Raspuns: Pentru x oarecare, adversarul calculeaza $H(x)$, o inverseaza si determina x' a.i $H(x') = H(x)$. Cu probabilitate mare $x' \neq x$ deci gaseste o a doua preimagine.

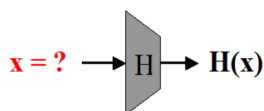
Atacuri în practică



$\{x\}$ = documente originale
 $\{x'\}$ = documente false
 cineva este de acord să semneze electronic documentul original, însă semnătura devine valabilă și pentru un document fals



documentul x care este semnat electronic poate fi înlocuit de documentul x'



dacă se cunoaște cheia de sesiune k_i calculată din cheia master k
 $x = H(k||i)$, atunci se determină cheia k

Atacul zilei de nastere

Analizăm posibilitatea de a determina o coliziune pornind de la un exemplu clasic: paradoxul nasterilor;

► Intrebare: Care este dimensiunea unui grup de oameni pentru ca 2 dintre ei sa fie nascuti in aceeași zi cu probabilitate $1/2$?

► Răspuns: 23!

► Generalizand, consideram o mulțime de dimensiune n si q elemente uniform aleatoare din aceasta multime $y_1, \dots, y_q$;

► Atunci pentru $q \geq 1.2 \times 2^{n/2}$ probabilitatea sa existe $i \neq j$ a.i. $y_i = y_j$ este $\geq 1/2$.

► Acest rezultat conduce imediat la un atac asupra funcțiilor hash cu scopul de a determina coliziuni:

- Adversarul alege $2^{n/2}$ valori x_i ;
- Calculează pentru fiecare $y_i = H(x_i)$;
- Cauta $i \neq j$ cu $H(x_i) = H(x_j)$;
- Dacă nu găsește nici o coliziune, reia atacul.

► Cum probabilitatea de succes a atacului este $\geq 1/2$, atunci numarul de incercari este ≈ 2 .

Transformarea Merkle-Damgard

Numim functie de compresie o functie hash rezistenta la coliziuni de intrare de lungime fixa

Intrebare: Intuitiv, ce se construiește mai usor, H_1 sau H_2 ?

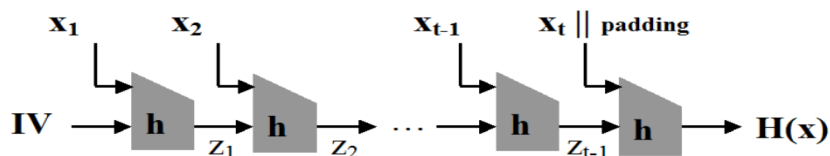
$$H_1 : \{0, 1\}^{m_1} \rightarrow \{0, 1\}^{n_1}, m_1 > n_1, m_1 \approx n_1$$

Raspuns: Cu cat domeniul si codomeniul functiei sunt mai apropiate ca dimensiune, numarul de coliziuni scade => coliziunile devin mai dificil de determinat

$$H_2 : \{0, 1\}^{m_2} \rightarrow \{0, 1\}^{n_2}, m_2 \gg n_2$$

Scopul este sa construim o functie hash pornind

de la o functie de compresie, pentru aceasta se aplica transformarea Merkle-Damgard



Notatii:

h = o functie de compresia de dimensiune fixa

$x = x_1 \parallel x_2 \parallel \dots \parallel x_{t-1} \parallel x_t$

lv = vector de initializare

padding = 100...0 || lungimea mesajului

Teorema: Daca h

prezinta rezistenta la
coliziuni, atunci si H
prezinta rezistenta la
coliziuni

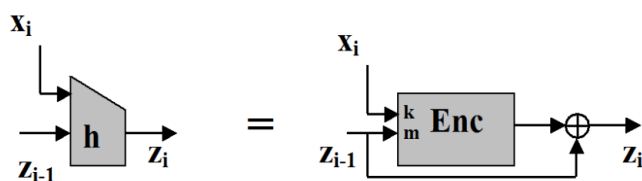
► Intuitiv, dacă există $x \neq x'$ a.î. $H(x) = H(x')$, atunci trebuie să existe $\langle z_{i-1}, x_i \rangle \neq \langle z'_{i-1}, x'_i \rangle$ a.î.
 $h(z_{i-1} || x_i) = h(z'_{i-1} || x'_i)$;

► Altfel spus, dacă se găsește o coliziune pentru H se găsește o coliziune și pentru h .

► Construcția Davies-Meyer:

Exemple de functii hash

- MD5: Nesigur din 1996, output pe 128 de biti
- SHA:
- SHA-0
- SHA-1 sunt nesigure;
- Sha-2 prezinta 3 variante: SHA-256 si SHA-512;
- SHA-3



► Enc este un sistem bloc care criptează z_{i-1} cu cheia x_i :

$$h(z_{i-1} || x_i) = Enc_{x_i}(z_{i-1}) \oplus z_{i-1}$$

Exercitii funcții hash

- Este $H(x) = x(mod N)$ o funcție hash rezistentă la coliziuni? Dacă da, explicați de ce, dacă nu, găsiți o coliziune.
- **Raspuns:** Funcția nu e rezistentă la coliziuni: pentru orice $x < N$ si orice $a \in \mathbb{N}$, x si $aN + x$ formeaza o coliziune.
- Este $H(x) = ax + b(mod N)$ cu $(a, N) = 1$ o functie hash rezistentă la coliziuni? Dacă da, explicați de ce, dacă nu, găsiți o coliziune.
- **Raspuns:** Funcția nu e rezistentă la coliziuni: cautam x_1 si x_2 asa incat $H(x_1) = H(x_2)$ adica $ax_1 + b = ax_2 + b(mod N)$. Obținem $a(x_1 - x_2) = 0(mod N)$. Cum (a, N) obținem ca $x_1 - x_2 = 0(mod N)$.

Curs 8.1 - SHA-3

- SHA-1: -output 160 biti
-atac pentru gasirea de coliziuni cu prefix - 2^{67} evaluari
-coliziuni cu prefix:
pornind de la prefixele P si P' , se cere gasirea mesajelor $M \neq M'$ cu $H(P || M) = H(P' || M')$
- SHA-2&3 safe

Function	Output Size	Security Strengths in Bits		
		Collision	Preimage	2nd Preimage
SHA-1	160	< 80	160	$160 - L(M)$
SHA-224	224	112	224	$\min(224, 256 - L(M))$
SHA-512/224	224	112	224	224
SHA-256	256	128	256	$256 - L(M)$
SHA-512/256	256	128	256	256
SHA-384	384	192	384	384
SHA-512	512	256	512	$512 - L(M)$
SHA3-224	224	112	224	224
SHA3-256	256	128	256	256
SHA3-384	384	192	384	384
SHA3-512	512	256	512	512
SHAKE128	d	$\min(d/2, 128)$	$\geq \min(d, 128)$	$\min(d, 128)$
SHAKE256	d	$\min(d/2, 256)$	$\geq \min(d, 256)$	$\min(d, 256)$

Keccak

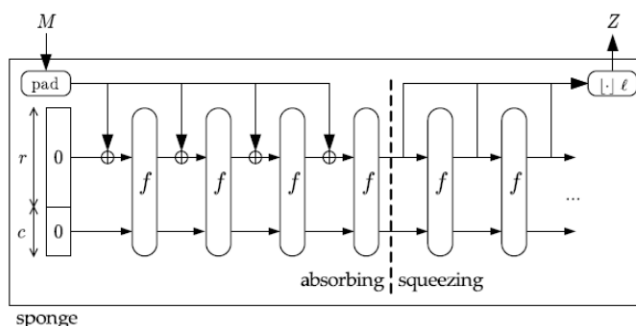
Castigatorul concursului SHA-3, a fost gandit sa difere complet de constructiile existente (AES, SHA-2). Foloseste sponge functions, principala componenta este permutarea f care accepta blocuri de 1600 biti

Notatii:

- r = bitrate
- c = capacity
- $b = c+r$ = width
- f = o permutare

Presupune 2 etape:

1. Absorbing phase: mesajul de intrare se sparge in blocuri de lungime r care se XOR-eaza la prima parte a stării, alternand cu aplicarea functiei f ;
2. Squeezing phase: partea superioara a starii este returnata la iesire, alternand cu aplicarea functiei f ; numarul de iteratii depinde de numărul de biti l necesari la iesire.



8.2 - Aplicatii ale functiilor hash

Retineti!

Cel mai bun atac generic pentru gasirea de coliziuni ne spune ca avem nevoie de functii hash cu output-ul pe $2n$ biti pentru securitate impotriva adversarilor care ruleaza in timp 2^n , spre deosebire de sistemele de criptare bloc, unde avem nevoie de n biti pentru securitate impotriva adversarilor care ruleaza in timp 2^n

Pentru un fisier x , $H(x)$ poate servi ca identificator unic (amprentă) - existenta unui fisier diferit cu același hash implică găsirea de coliziuni in H). Prezintam cateva aplicatii care decurg de aici:

- Amprentarea virusilor
 - scanner-ele de virusi pastreaza hash-urile virusilor cunoscuti
 - verificarea fisierelor ptential malitioase se face comparand hash-ul lor cu hash-ul virusior cunoscuti
- Deduplicarea datelor - stocarea in cloud partajata de mai multi utilizatori - se verifica daca hash-ul unui fisier nou (de incarcat) exista deja in cloud, caz in care se adauga doar un pointer la fisierul respectiv

Stocarea parolelor

O greseala frecventa de implementare la metoda de autentificare bazata pe nume de utilizator si parola sunt mesaje de tipul: Nume de utilizator inexistent/Parola gresita. Aceste mesaje ofera informatii suplimentare adversarului

Intrebare: De ce parolele NU trebuie stocate in clar

Raspuns: Pentru ca daca adversarul capata acces la fisierul de parole atunci afla direct parolele tuturor utilizatorilor!

Pentru stocarea parolelor se folosesc functii hash. In fisierul de parole se stocheaza, pentru fiecare utilizator perechi de forma (username, $H(pwd)$). Functiile hash sunt one-way, cunoscand $H(pwd)$ nu se poate determina pwd .. Aceasta metoda de stocare are avantajul ca nu ofera adversarului acces direct la parole, chiar daca acesta detine fisierul de parole.

Atac de tip dictionar

Adversarul poate sa verifice pe rand toate parolele cu probabilitate mare de aparitie(pecum parole uzuale). Acestea formeaza termenii unui dictionar. Pentru a minimiza sansele unor astfel de atacuri:

- ▶ se blocheaza procesul de autentificare după un anumit numar de incercari nereusite;
- ▶ se obliga utilizatorul sa folosească o parola care satisface anumite criterii : o lungime minima, utilizarea a cel puțin 3 tipuri de simboluri (litere mici, litere mari, cifre, caractere speciale);
- ▶ In caz de succes, adversarul determina parola unui singur utilizator
- ▶ Pentru a determina parolele mai multor utilizatori simultan, un adversar poate precalcula valorile hash ale parolelor din dicționar;
- ▶ Daca adversarul capata acces la fisierul de parole, atunci verifica valorile care se regăsesc în lista precalculata;
- ▶ Toate conturile utilizatorilor pentru care se potrivesc valorile sunt compromise;
- ▶ Atacul necesită capacitate de stocare mare: trebuie stocate toate perechile (pwd, H(pwd)) unde pwd este o parolă din dicționar;

Rainbow tables

- ▶ Rainbow tables introduc un compromis intre capacitatea de stocare si timp;
- ▶ Se compun lanțuri de lungime t: $\text{pwd}_1 \rightarrow H(\text{pwd}_1) \rightarrow f \text{pwd}_2 \rightarrow H(H(\text{pwd}_2)) \rightarrow f \dots \rightarrow f \text{pwd}_{t-1} \rightarrow H(H(\text{pwd}_{t-1})) \rightarrow f \text{pwd}_t \rightarrow H(H(\text{pwd}_t))$
- ▶ f este o functie de mapare a valorilor hash in parole;
- ▶ Atentie! Funcția f nu este inversa funcției H (s-ar pierde proprietatea de unidirectionalitate a funcției hash)
- ▶ Se memorează doar capetele lanțurilor, valorile intermediare se genereaza la nevoie;
- ▶ Dacă T este suficient de mare, atunci capacitatea de stocare scade semnificativ;
- ▶ Algoritmul de determinare a unei parole:
 1. Se cauta valoarea hash a parolei in lista de valori hash a tabelii stocate;
 - 1.1 Daca se găsește, atunci lanțul cautat este acesta si se trece la pasul 2;
 - 1.2 Daca nu se găsește, se reduce valoarea hash prin funcția de reducere f intr-o parola căreia i se aplica functia H si se reia cautarea de la pasul 1;
 2. Se genereaza intreg lantul plecand de la valoarea initiala stocata. Parola corespunzatoare este cea situata in lant, nainte de valoarea hash cautata.

Salting

Pentru a evita atacurile precedente se foloseste salting. La inregistrare, se stocheaza pt fiecare utilizator (username,salt, H(pwd||salt)). Salt este o secventa aleatoare de n biti distincta pt fiecare utilizator. Adversarul nu poate precalcula valorile in hash inainte de a obtine acces la fisierul de parole decat daca foloseste 2^n valori posibile salt pt fiecare parola. Atacurile devin deci impracticabile pentru n suficient de mare. Un alt avantaj este ca 2 useri cu parole identice le vor avea stocate diferit

Parole cu unica folosinta

- ▶ Cazul extrem de schimbare periodică a parolei îl reprezintă parolele de unică folosință;
- ▶ Utilizatorul folosește o listă de parole, la fiecare logare utilizând următoarea parolă din listă;
- ▶ Această listă se calculează pornind de la o valoare x folosind o funcție hash H;
- ▶ Să considerăm o listă de 3 parole de unică folosință:

$$P_0 = H(H(H(x)))$$

$$P_1 = H(H(x))$$

$$P_2 = H(x)$$

$$P_3 = x$$
- ▶ La înregistrare, în fișierul de parole se păstrează tripletul: $(\text{username}, 1, P_0 = H(H(H(x))))$
- ▶ Utilizatorul introduce o parolă P_1 și autentificarea se realizează cu succes dacă $H(P_1) = P_0$;
- ▶ Se actualizează fișierul de parole cu noua parolă introdusă: $(\text{username}, 2, P_1 = H(H(x)))$
- ▶ Procesul continuă până se ajunge la $H(x)$;
- ▶ Fiind cunoscută o parolă din secvență, se poate calcula imediat parola anterioară, dar NU se poate calcula parola următoare.

8.3 - Criptografie cu cheie publica (asimetrica)

Limitările criptografiei simetrice

Problema 1: Distribuirea cheilor

Ele se pot distribui ori printr-un canal de comunicare nesecurizat (ceea ce nu e ok) sau se transmit printr-un canal de comunicare sigur care presupune un serviciu de mesagerie de încredere - posibil doar la nivel guvernamental sau militar

Problema 2: Stocarea secretă a cheilor

Este nevoie de $n(n-1)/2$ chei pentru ca fiecare 2 angajați să poată comunica criptat (la o companie cu n angajați). La acestea se adaugă și cheile pentru accesul la resurse. Apare astfel o problemă de logistică, foarte multe chei sunt dificil de menținut și utilizat, mai mult, cu cât sunt mai multe chei cu atât sunt mai dificil de stocat safe, deci cresc șansele de a fi furate.

Totuși dacă numărul de chei este mic, există soluții de stocare cu securitate crescută; Un exemplu îl reprezintă dispozitivele de tip smartcard; Acestea realizează calculele criptografice folosind cheia stocată, asigurând faptul că niciodată cheia secretă nu ajunge în calculator.

Problema 3: Medii de comunicare deschise

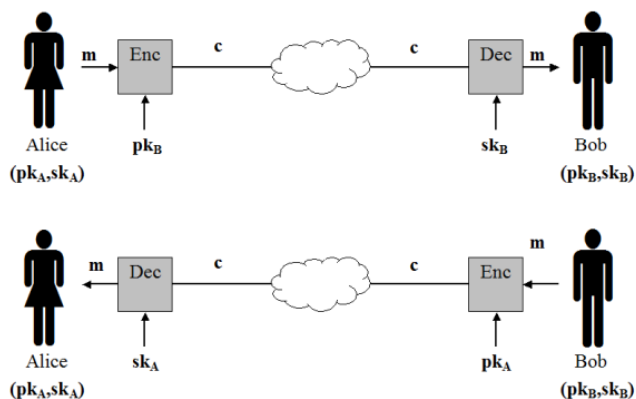
Criptografia simetrică este insuficientă în medii deschise în care participanții nu se întâlnesc niciodată (ex: o tranzacție sau un e-mail unei persoane necunoscute)

Problema 4: Imposibilitatea non-repudierii

O cheie simetrică este deținută de cel puțin 2 părți; Este imposibil de demonstrat de exemplu că un MAC a fost produs de una dintre cele 2 părți comunicante; De aceea nu se poate utiliza autentificarea simetrică pentru a atesta sursa unui mesaj sau document.

Sisteme de criptare asimetrice

Folosesc chei diferite pentru criptare și decriptare



Definiție

Un sistem de criptare asimetric definit peste $(\mathcal{M}, \mathcal{C})$, cu:

- ▶ $\mathcal{M}$ = spațiul textelor clare (mesaje)
- ▶ $\mathcal{C}$ = spațiul textelor criptate

este un triplet $(\text{Gen}, \text{Enc}, \text{Dec})$, unde:

1. $\text{Gen}(1^n)$ - generează cheile (pk, sk)
2. Enc - primește la intrare o cheie publică pk și un mesaj m și calculează $c \leftarrow \text{Enc}_{pk}(m)$
3. Dec - primește la intrare o cheie secretă pk și un mesaj criptat c și întoarce mesajul clar sau eroare (simbolul $\perp$)

a.î. $\forall m \in \mathcal{M}$, (pk, sk) generate cu algoritmul $\text{Gen}(1^n)$
 $\text{Dec}_{sk}(\text{Enc}_{pk}(m)) = m$.

Avantaje Criptografie asimetrica

Număr mic de chei, simplifică problema distribuirii cheilor, fiecare participant trebuie să stocheze o cheie secretă de lungă durată, permite comunicarea sigură pe canale publice, rezolvă problema mediilor de comunicare deschise.

Dezavantaje criptografie asimetrica:

Este mult mai lentă decât cea simetrică, compromiterea cheii private conduce la compromiterea tuturor mesajelor criptate

- ▶ Spre deosebire de criptarea simetrică, criptarea asimetrică folosește o pereche de chei:
- ▶ Cheia publică pk este folosită pentru criptare;
- ▶ Cheia secretă sk este folosită pentru decriptare;
- ▶ Cheia publică este larg răspândită pentru a asigura posibilitatea de criptare oricui dorește să transmită un mesaj către entitatea căreia îi corespunde;
- ▶ Cheia secretă este privată și nu se cunoaște decât de entitatea căreia îi corespunde;
- ▶ Considerăm (pentru simplitate) că ambele chei au lungime cel puțin n biți.

permite, indiferent de sursa, necesita verificarea autenticitatii cheii publice.

Important

Criptografia simetrica NU rezolva toate problemele criptografiei
Criptografia asimetrica apare in completarea criptografiei simetrice

Criptografia asimetrică vs. Criptografia simetrică

Criptografia simetrică

- ▶ necesită secretizarea întregii chei
- ▶ folosește aceeași cheie pentru criptare și decriptare
- ▶ rolurile emițătorului și ale receptorului pot fi schimbate
- ▶ pentru ca un utilizator să primească mesaje criptate de la mai mulți emițători, trebuie să partajeze cu fiecare câte o cheie

Criptografia asimetrică

- ▶ necesită secretizarea unei jumătăți din cheie
- ▶ folosește chei distincte pentru criptare și decriptare
- ▶ rolurile emițătorului și ale receptorului nu pot fi schimbate
- ▶ o pereche de chei asimetrice permite oricui să transmită informație criptată către entitatea căreia îi corespunde

8.4 - Teoria numerelor pentru criptografie

▶ $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$

▶ $\mathbb{N} = \{0, 1, 2, \dots\}$

▶ $\mathbb{Z}_+ = \{\dots, -2, -1, 0, 1, 2, \dots\}$

▶ Pentru $a, n \in \mathbb{N}$ notăm $\gcd(a, n)$ ca fiind cel mai mare divizor comun (greatest common divisor) al lui a și n .

▶ Exemplu: $\gcd(30, 50) = 10$.

▶ $a = qn + r$ cu $0 \leq r < n$.

Considerăm împărțirea lui a la n .

Atunci q este catul împărțirii iar r este restul și notăm

$$a \bmod n = r$$

▶ pentru $n \in \mathbb{Z}_+$ notăm

▶ $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$

▶ $\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n \mid \gcd(a, n) = 1\}$

▶ $\phi(n) = |\mathbb{Z}_n^*|$

▶ Dacă $n \in \mathbb{Z}_+$ atunci $G = \mathbb{Z}_n^*$ împreună cu operația "*" definită

$$a * b = ab \bmod n$$

pentru $a, b \in G$ formează un grup și are cele trei proprietăți:

- ▶ asociativitatea: operația $*$ este asociativă
- ▶ element neutru: există un element $1 \in G$ așa încât $a * 1 \bmod n = 1 * a \bmod n = a, \forall a \in G$.
- ▶ element inversabil: pentru orice $a \in G$ exista un unic $b \in G$ așa încât $a * b = b * a = 1 \bmod n$.
 b se numeste inversul lui a și îl notăm cu $a^{-1} \bmod n$.

Scurtături computaționale

▶ calculați $5 * 8 * 10 * 16 \bmod 21$.

▶ Prima variantă: Calculăm mai întâi $5 * 8 * 10 * 16 = 6400$ și apoi calculăm $6400 \bmod 21 = 16$

▶ A doua variantă (mai rapidă):

▶ $5 * 8 \bmod 21 = 40 \bmod 21 = 19$

▶ $19 * 10 \bmod 21 = 190 \bmod 21 = 1$

▶ $1 * 16 \bmod 21 = 16$

Ordinul unui grup

- Ordinul unui grup G este numărul de elemente din acel grup, îl notăm cu $|G|$.

- **Exemplu:** Ordinul lui $\mathbb{Z}_{21}^* = 12$ pentru că

$$\mathbb{Z}_{21}^* = \{1, 2, 4, 5, 8, 10, 11, 13, 16, 17, 19, 20\}$$

- Fie G un grup de ordin m și $a \in G$. Atunci:

- $a^m = 1$.

- Pentru orice $i \in \mathbb{Z}$, $a^i = a^{i \bmod m}$.

- **Exemplu:** Calculați $5^{74} \bmod 21$.

- **Răspuns:** Fie $\mathbb{Z}_{21}^*$ și $a = 5$. Atunci $m = 12$ și

- $5^{74} \bmod 21 = 5^{74 \bmod 12} \bmod 21 = 5^2 \bmod 21 = 4$.

8.5 - Prezumpții criptografice dificile

Prezumpții criptografice dificile

- Criptografia modernă se bazează pe prezumpția că *anumite* probleme nu pot fi rezolvate în timp polinomial;
- Până acum am văzut că schemele de criptare și de autentificare se bazează pe prezumpția existenței permutărilor pseudoaleatoare;
- Dar această prezumpție e nenaturală și foarte puternică;
- În practică, PRF pot fi instanțiate cu cifruri bloc;
- Însă metode pentru a demonstra pseudoaleatorismul construcțiilor practice relativ la alte prezumpții "mai rezonabile" nu se cunosc;
- Dar e posibil a demonstra existența permutărilor pseudoaleatoare pe baza unei prezumpții mult mai slabe, cea a existenței funcțiilor one-way;
- În continuare vom introduce câteva două considerate "dificile": **problema factorizării** și **problema logaritmului discret** și vom prezenta funcții conjecturate ca fiind one-way bazate pe aceste probleme;
- Tot materialul ce urmează se bazează pe noțiuni de teoria numerelor;
- La criptografia *simetrică* (cu cheie secretă) am văzut primitive criptografice (i.e. funcții hash, PRG, PRF) care pot fi construite eficient fără a implica teoria numerelor;
- La criptografia *asimetrică* (cu cheie publică) construcțiile cunoscute se bazează pe probleme matematice dificile din teoria numerelor;

Problema factorizării

Fiind dat un număr compus N , problema cere să se găsească două numere prime x_1 și x_2 pe n biți așa încât $N = x_1 \cdot x_2$; Cele mai greu de factorizat sunt numerele care au factori primi foarte mari.

Generarea numerelor prime

Pentru a putea folosi problema în criptografie, trebuie să generăm numere prime aleatoare în mod eficient. Putem genera un număr prim aleator pe n biți prin alegerea repetată de numere aleatoare pe n biți până când găsim unul prim

Teoremă

Pentru orice $n > 1$, proporția de numere prime pe n biți este de cel puțin $1/3n$.

- Rezultă imediat că dacă testăm $t = 3n^2$ numere, probabilitatea ca un număr prim să nu fie ales este cel mult e^{-n} , deci neglijabilă.
- Deci avem un algoritm în timp polinomial pentru găsirea numerelor prime

Pentru eficiența ne interesează 2 aspecte:

1. Probabilitatea ca un nr aleator pe n biți să fie prim
2. Cum testăm eficient ca un număr dat p este prim

Pentru distribuția numerelor prime, se cunoaște următorul rezultat matematic:

Testarea primalității

Cei mai eficienți algoritmi sunt probabilisti.

Dacă numărul p dat este prim atunci algoritmul întotdeauna întoarce un rezultat prim

Dacă p este compus atunci cu probabilitate mare algoritmul va întoarce compus

Concluzie: Dacă output-ul este compus atunci p sigur este compus, dacă output-ul este prim atunci cu probabilitate mare p este prim dar este posibil să se fi produs o eroare.

Algoritmi de factorizare

Deocamdată nu se cunosc algoritmi polinomiali pentru problema factorizării.

- **Reamintim:** Fiind dat un număr compus N , **problema factorizării** cere să se găsească 2 numere prime p și q a.î. $N = pq$;
- Considerăm $|p| = |q| = n$ și deci $n = O(\log N)$;
- Metoda cea mai simplă este împărțirea numărului N prin toate numerele p impare din intervalul $p = 3, \dots, \lfloor \sqrt{N} \rfloor$.
- Complexitatea timp este $O(\sqrt{N} \cdot (\log N)^c)$ unde c este o constantă, adică exponențială în numărul b de biți al lui N ; $b = \log_2 N$. Complexitatea este $O(N^{1/2}) = O(2^{b/2})$.
- Pentru $N < 10^{12}$ metoda este destul de eficientă.

Experimentul logaritmului discret $DLog_A(n)$

1. Generează $(\mathbb{G}, q, g)$ unde $\mathbb{G}$ este un grup ciclic de ordin q (cu $|q| = n$) iar g este un generator al lui $\mathbb{G}$.
2. Alege $h \leftarrow^R \mathbb{G}$. (se poate alege $x' \leftarrow^R \mathbb{Z}_q$ și apoi $h := g^{x'}$.)
3. $\mathcal{A}$ primește $\mathbb{G}, q, g, h$ și întoarce $x \in \mathbb{Z}_q$;
4. Output-ul experimentului este 1 dacă $g^x = h$ și 0 altfel.

Definiție

Spunem că problema logaritmului discret (DLP) este dificilă dacă pentru orice algoritm PPT $\mathcal{A}$ există o funcție neglijabilă negl așa încât

$$\Pr[DLog_{\mathcal{A}}(n) = 1] \leq \text{negl}(n)$$

Problema logaritmului discret

- O altă prezumție dificilă este DLP (Discrete Logarithm Problem) (sau PLD (Problema Logaritmului Discret));
 - Considerăm $\mathbb{G}$ un grup ciclic de ordin q ;
 - Există un generator $g \in \mathbb{G}$ a.î. $\mathbb{G} = \{g^0, g^1, \dots, g^{q-1}\}$;
 - Echivalent, pentru fiecare $h \in \mathbb{G}$ există un *unic* $x \in \mathbb{Z}_q$ a.î. $g^x = h$;
 - x se numește **logaritmul discret** al lui h în raport cu g și se notează
- $$x = \log_g h$$
- Există câteva clase de grupuri ciclice pentru care DLP este considerată dificilă;
 - Una dintre ele este clasa grupurilor ciclice de *ordin prim* (în aceste grupuri, problema este "cea mai dificilă");
 - DLP nu poate fi rezolvată în timp polinomial în grupurile care nu sunt de ordin prim, ci doar este *mai ușoară*;
 - În aceste grupuri căutarea unui generator și verificarea că un număr dat este generator sunt triviale.

- DLP este considerată dificilă în grupuri ciclice de forma $\mathbb{Z}_p^*$ cu p prim;
- Însă pentru $p > 3$ grupul $\mathbb{Z}_p^*$ NU are ordin prim;
- Aceasta problemă se rezolvă folosind un *subgrup* potrivit al lui $\mathbb{Z}_p^*$;

De retinut:

- Cel mai rapid algoritm de factorizare necesita timp sub-exponential
- Problema logaritmului discret este dificila

Curs 9.1 - Notiuni de securitate in criptografia asimetrica

Criptografia asimetrica

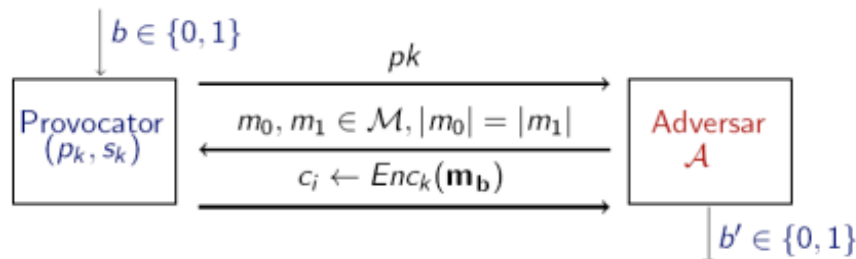
- Definitia analog cu criptografia simetrica
- Adversarul cunoaste atat textul criptat, cat si cheia publica

Securitatea perfecta NU este posibila in cadrul criptografiei cu cheie publica (indiferent de lungimea cheilor si a mesajelor)

- avand cheia pk si un text criptat $c = Enc_{pk}(m)$, un adversar nelimitat computational poate determina mesajul m cu probabilitate 1

Indistinctibilitatea in criptografia cu cheie publica este corespondenta notiunii similare din criptografia cu cheie secreta

- Vom defini aceasta notiune pe baza unui *experiment* de indistinctibilitate $PubK^{eav}_{A,\pi}(n)$ unde $\pi = (Enc, Dec)$ este schema de criptare iar n este parametrul de securitate al schemei π
- Personaje participante: *adversarul* A care incearca sa sparga schema si un *challenger*
- *Experimentul* $PubK^{eav}_{A,\pi}(n)$



- Output-ul experimentului este 1 daca $b' = b$ si 0 altfel. Daca $PubK^{eav}_{A,\pi}(n) = 1$, spunem ca A a efectuat experimentul cu succes

Securitate pentru interceptare simpla

- O schema de criptare $\pi = (Enc, Dec)$ este indistinctibila in prezenta unui atacator pasiv daca pentru orice adversar A exista o functie neglijabila $negl$ a.i.
 $Pr[PubK^{eav}_{A,\pi}(n) = 1] \leq 1/2 + negl(n)$
- Principala diferenta fata de definitia similara studiata la criptografia cu cheie secreta este ca A primeste cheia publica pk
- Adica A primeste acces gratuit la un oracol de criptare, ceea ce inseamna ca el poate calcula $Enc_{pk}(m)$ pentru orice m
- Prin urmare, definitia este echivalenta cu cea pentru securitate CPA (nu mai este necesar oracolul de criptare pentru ca A isi poate cripta singur mesajele)
- Reamintim ca in criptografia simetrica exista scheme indistinctibil sigure, dar care NU sunt CPA-sigure

Insecuritatea schemelor deterministe

- O schema determinista nu poate fi CPA-sigura !!!
- CPA-secure = indistinctibilitate pentru interceptare simpla (in criptografia asimetrica)
- **Teorema:** Nicio schema de criptare cu cheie publica determinista NU poate fi semantic sigura pentru interceptarea simpla

Securitate CCA

- Notiunea de securitate CCA ramane identica cu cea de la sistemele simetrice
- Capabilitatile adversarului:
 - poate interactiona cu un oracol de decriptare, fiind un adversar activ care poate rula atacuri in timp polinomial;
 - poate transmite catre oracolul de decriptare anumite mesaje c si primeste inapoi mesajul clar corespunzator;
 - adversarul nu mai necesita acces la oracolul de criptare pentru ca detine cheia publica pk si poate realiza singur criptarea oricarui mesaj m

Curs 9.2 - Criptare hibrida

- Criptarea cu cheie secreta este mult mai rapida decat criptarea cu cheie publica
- Pentru mesajele care sunt suficient de lungi, se foloseste criptare cu *cheie secreta* in tandem cu criptarea cu *cheie publica* => *criptare hibrida*

Pentru criptarea unui mesaj m , se urmeaza doi pasi:

1. Expeditorul alege aleator o cheie k pe care o cripteaza folosind cheia publica a destinatarului, rezultand $c1 = Enc_{pk}(k)$
 - a. destinatarul va putea decripta k
 - b. k secreta pentru un adversar
2. Expeditorul cripteaza m folosind o schema de criptare cu cheie secreta (Enc' , Dec') cu cheia k , rezultand $c2 = Enc'_k(m)$

=> mesajul criptat este $c = (c1, c2)$

Teorema: Daca Π este o schema de criptare cu cheie publica CPA-sigura iar Π' este o schema de criptare cu cheie secreta sigura semantic, atunci constructia hibrida Π^{hyb} este o schema de criptare cu cheie publica CPA-sigura.

- Este suficient ca Π' sa satisfaca notiunea mai slaba de securitate semantica (care nu implica securitate CPA) deoarece cheia secreta k este una 'nouă' si aleasa aleator de fiecare data cand se cripteaza un mesaj;
- Cum o cheie k este folosita o singura data, e suficienta notiunea de securitate la interceptare simpla pentru securitatea schemei hibride.

Curs 9.3 - RSA

Functii one-way

- O functie $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ este *one-way* daca urmatoarele doua conditii sunt indeplinite:
 1. **Usor de calculat:** Exista un algoritm polinomial pentru calculul lui f

2. **Dificil de inversat:** Pentru orice algoritm polinomial A , exista o functie neglijabila negl asa incat $\Pr[\text{Invert}_A, f(n) = 1] \leq \text{negl}(n)$
- Reprezinta o primitiva criptografica minima, necesara si suficienta pentru criptarea cu cheie secreta dar si pentru codurile de autentificare a mesajelor
 - **Problema factorizarii** numerelor mari in produs de doua numere prime de aceeasi lungime este one-way, insa nu poate fi folosita direct pentru criptografie.

Problema RSA

- Problema RSA se bazeaza pe dificultatea factorizarii numerelor mari: $N = p \cdot q$, p si q prime
- Fie $\mathbb{Z}_N^*$ un grup de ordin $\phi(N) = (p - 1)(q - 1)$
- Daca se cunoaste factorizarea lui N , atunci $\phi(N)$ este usor de calculat
- Fixam e cu $\gcd(e, \phi(N)) = 1$. Atunci $(x^e)^d = x^{ed} \bmod \phi(N) = x \bmod N = \bmod N = (x^d)^e$
- x^d se numeste radacina de ordin e a lui x modulo N
- Daca p si q se cunosc, atunci putem calcula $\phi(N)$ si $d = e^{-1} \bmod \phi(N)$
- Daca p si q nu se cunosc
 - calculul lui $\phi(N)$ este la fel de dificil precum factorizarea lui N
 - calculul lui d este la fel de dificil precum factorizarea lui N

Experimentul RSA - $\text{inv}_{A, \text{GenRSA}(n)}$

- Problema RSA este dificila cu privire la GenRSA daca pentru orice algoritm PPT A exista o functie neglijabila negl a.i. $\Pr[\text{RSA} - \text{inv}_{A, \text{GenRSA}(n)} = 1] \leq \text{negl}(n)$
- Consideram algoritmul $\text{GenRSA}(1^n)$ care are ca output (N, e, d) unde $ed = 1 \bmod \phi(N)$
- Consideram experimentul RSA pentru un algoritm A si un parametru n
 - Executa GenRSA si obtine (N, e, d)
 - Alege $y \leftarrow \mathbb{Z}_N^*$
 - A primeste N, e, y si intoarce $x \in \mathbb{Z}_N^*$
 - Output-ul experimentului este 1 daca $x = y \bmod N$ si 0 altfel

GenRSA

- Presumptia RSA este ca exista un algoritm GenRSA pentru care problema RSA este dificila
- Un algoritm GenRSA poate fi construit pe baza unui numar compus impreuna cu factorizarea lui

Algorithm 1 GenRSA

Input: n

Output: N, e, d

- 1: genereaza p și q prime pe n -biti; $N = p \cdot q$
 - 2: $\phi(N) = (p - 1)(q - 1)$
 - 3: gaseste e a.i. $\gcd(e, \phi(N)) = 1$
 - 4: calculează $d := e^{-1} \bmod \phi(N)$
 - 5: return N, e, d
-

- Valoarea lui e aleasa pare ca nu afecteaza dificultatea problemei RSA
- In practica se foloseste $e \in \{3, 16\}$ pentru exponentiere eficienta

- Daca N este usor de factorizat, atunci problema RSA este usoara
- Pentru ca problema RSA sa poata sa fie dificila, trebuie ca N -ul ales in GenRSA sa fie dificil de factorizat in produs de doua numere prime
- Nu se cunoaste nici o dovada ca nu exista o alta metoda de a rezolva problema RSA care sa nu implice calculul lui $\phi(N)$ sau al lui d
- **Exemplu**
 - Presupunem $(N, p, q) = (143, 11, 13)$. Atunci $\phi(N) = 120$
 - Alegem e asa incat $\gcd(e, \phi(N)) = 1$, fie $e = 7$
 - Calculam $d = e^{-1} \bmod \phi(N)$ si obtinem $d = 103$. Output-ul GenRSA este $(143, 7, 103)$

Textbook RSA

- Definim sistemul de criptare Textbook RSA pe baza problemei prezentata anterior
 1. Se ruleaza GenRSA pentru a determina N, e, d
 - a. Cheia publica este: $pk = (N, e)$
 - b. Cheia privata este $sk = d$
 2. Enc: data o cheie publica (N, e) si un mesaj $m \in \mathbb{Z}_N$, intoarce $c = m^e \bmod N$
 3. Dec: data o cheie secreta (N, d) si un mesaj criptat $c \in \mathbb{Z}_N$, intoarce $m = c^d \bmod N$
- Sistemul de criptare este corect pentru ca $\text{Dec}_{sk}(\text{Enc}_{pk}(m)) = m$ astfel: $(m^e)^d \bmod N = m^{ed \bmod \phi(N)} \bmod N = m^1 \bmod N = m$

Problema: Determinismul

- *Intrebare:* Este Textbook RSA CPA-sigur sau CCA-sigur?
- *Raspuns:* **NU!** Sistemul este determinist, deci nu poate rezista definitiilor de securitate!

Problema: Utilizarea multipla a modulului

- Cunoscand e, d, N cu $(e, \phi(N)) = 1$ se poate determina eficient factorizarea lui N
- *Intrebare:* Este corect sa se utilizeze mai multe perechi de chei care folosesc acelasi modul?
- *Raspuns:* NU! Fie 2 perechi de chei:
 - $pk_1 = (N, e_1); sk_1 = (N, d_1)$
 - $pk_2 = (N, e_2); sk_2 = (N, d_2)$
- Posesorul perechii (pk_1, sk_1) factorizeaza N , apoi determina $d_2 = e_2^{-1} \bmod \phi(N)$

Curs 9.4 - PKCS

Padded RSA

- Am vazut ca *Textbook RSA* este nesigur
 - Eliminam una dintre problemele principale, aceea este ca sistemul este determinist
 - Introducem in acest sens *Padded RSA*
 - Ideea este sa se adauge un numar aleator (pad) la mesajul clar inainte de criptare
 - Notam in continuare cu n parametrul de securitate (conform GenRSA)
1. Se ruleaza *GenRSA* pentru a determina N, e, d
 - a. Cheia publica este: (N, e)

- b. Cheia privata este (N, d)
2. *Enc*: data o cheie publica (N, e) si un mesaj $m \in \{0, 1\}^{l(n)}$, alege $r \xleftarrow{R} \{0, 1\}^{l(n)-1}$, interpreteaza $r||m$ ca un element in $\mathbb{Z}_N$ si intoarce $c = (r||m)^e \bmod N$
 3. *Dec*: data o cheie secreta (N, d) si un mesaj criptat $c \in \mathbb{Z}_N$, calculeaza $c^d \bmod N$ si intoarce ultimii $l(n)$ biti
- Pentru $l(n)$ foarte mare, atunci este posibil un atac prin forta bruta care verifica toate valorile posibile pentru r
 - Pentru $l(n)$ mic se obtine securitate CPA:
 - **Teorema**: Daca problema RSA este dificila, atunci Padded RSA cu $l(n) = O(\log n)$ este CPA-sigura.

PKCS

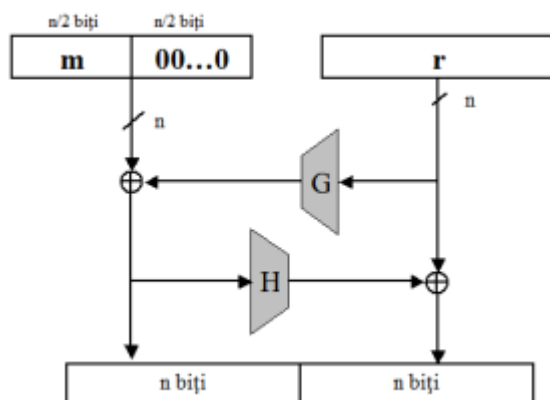
- Foloseste Padded RSA
- Utilizat in HTTPS, SSL/TLS, XML Encryption
- Notam k lungimea modulului N in bytes: $2^{8(k-1)} \leq N < 2^{8k}$
- Mesajele m care se cripteaza se considera multiplii de 8 biti, de maxim $k - 11$ bytes
- Criptarea se realizeaza astfel: $(00000000||00000010||r||00000000||m)^e \bmod N$
- r este ales aleator, pe $k - D - 3$ bytes nenuli, unde D este lungimea lui m in bytes;

Securitatea PKCS #1 v1.5

- Se crede ca este CPA-sigur, dar acest lucru nu este demonstrat
- Cu siguranta insa nu este CCA-sigur
- Scopul adversarului este sa decripteze un text c :
 - Adversarul transmite catre server $c' = r^e * c \bmod N$
 - Raspunsul serverului indica adversarului daca c' este valid (i.e. incepe cu 02)
 - Adversarul foloseste raspunsul primit pentru a determina informatii despre m
 - Repeta atacul pana determina mesajul m

OAEP

- OAEP este o modalitate de padding
- OAEP este o metoda nedeterminista si inversabila care transforma un mesaj m de lungime $n/2$ intr-o secventa m' de lungime $2n$
- Notam $m' = \text{OAEP}(m, r)$, unde r este o secventa aleatoare (de lungime n)
- RSA-OAEP *cripteaza* $m \in \{0, 1\}^{n/2}$ folosind cheia publica (N, e) ca: $(\text{OAEP}(m, r))^e \bmod N$
- RSA-OAEP *decripteaza* c folosind cheia secreta (N, d) ca: $(m, r) = \text{OAEP}^{-1}(c^d \bmod N)$



Utilizarea RSA in practica

In practica se folosesc **PKCS #1 v1.5** si **PKCS #1 v2.0(OAEP)**

Notatii:

- gcd = greatest common divisor
- $\phi(N)$ = factorizarea lui N
- pk / sk = public key / secret key
- PKCS = Public-Key Cryptography Standard
- OAEP = Optimal Asymmetric Encryption Standard
- G, H = functii hash
- $m \in \{0, 1\}^{n/2}$ = mesaj

Curs 10.2 - Problema logaritmului discret(PLD)

PLD - din curs 8.5

Algoritmi pentru calculul logaritmului discret

- Problema PLD se poate rezolva, desigur, prin forta bruta, calculand pe rand toate puterile x ale lui g pana cand se gaseste una potrivita pentru care $g^x = h$
- Complexitatea timp este $O(q)$ iar complexitatea spatiu este $O(1)$
- Daca se precalculeaza toate valorile (x, g^x) cautarea se face in timp $O(1)$ si spatiu $O(q)$
- Sunt de interes algoritmi care pot obtine un timp mai bun la rulare decat prin forta bruta, realizand un compromis spatiu-timp
- Se cunosc mai multi astfel de algoritmi impartiti in doua categorii
 - Algoritmi *generici* care functioneaza in grupuri arbitrare
 - Algoritmi *non-generici* care lucreaza in grupuri specifice - exploateaza prop. Specifice

Metoda Baby-step/Giant-step - algoritmi generici

- Creata de Shanks, calculeaza logaritmul discret intr-un grup de ordin q in $O(\sqrt{q})$ timp si $O(\sqrt{q})$ spatiu
- Pentru g apartine $\text{mathbb{G}}$ generatiu, elementele lui G sunt
- Stim ca $h = g^x$ se afla intre aceste valori
- Descrierea metodei
 - Marcam si memoram anumte puncte din grup aflata la distanta $t = \sqrt{q}$ - giant-step
 - Stim ca $h = g^x$ se afla in unul din aceste intervale
 - Calculand cu baby steps valorile $h \cdot g^1, h \cdot g^2, \dots, h \cdot g^t$
 - Una din ele va fi egala cu unul din puncte marcate - $h \cdot g^i = g^{k \cdot t}$

- Complexitatea timp este $\mathcal{O}(\sqrt{q} \cdot \text{polylog}(q))$ iar complexitatea spațiu este $\mathcal{O}(\sqrt{q})$
- Este **optima** ca timp de rulare dar exista alg mai eficienti ca spatiu
- Algoritmul **Polig-Hellman** poate fi folosit atunci cand se cunoaste factorizarea ordinului q al grupului iar timpul de rulare depinde de factori primi ai lui q . Pentru ca algoritmul sa nu fie eficient trebuie ca cel mai mare factor prim al lui q sa fie de ordinul 2^{160}

Algoritmi non-generici pentru calculul logaritmului discret

- Potential mai puternici decat cei generici
- Cel mai cunoscut in $\mathbb{Z}_p^*$ cu p prim este GNFS - General Number Field Sieve cu complexitate timp $2^{\mathcal{O}(n^{1/3} \cdot (\log n)^{2/3})}$ unde $|p| = \mathcal{O}(n)$;
- Exista si **metoda de calcul a indicelui** care rezolva DLP in grupuri ciclice $\mathbb{Z}_p$ cu p prim in timp sub-exponential in lungimea lui p . Seamana cu algoritmul sitei patratice pentru factorizare

Important de retinut

- Cel mai bun algoritm pentru DLP este sub-exponential
- Se pot contrui functii hash rezistente la coliziuni bazate pe dificultatea DLP

Curs 10.3 - Schimbul de chei Diffie-Hellman

- Semnaturile digitale sunt analogul MAC-urilor din criptografia simetrica(corespondent digital al unei semnaturi reale)
- Schimbul de chei se introduce pentru a facilita introducerea sistemelor de criptare bazate pe DLP

Conceptul de schimb de chei

- Un protocol de schimb de chei este un protocol prin care 2 persoane care nu partajeaza in prealabil niciun secret pot genera o cheie comuna, secreta
- Comunicarea necesara pentru stabilirea cheii se realizeaza printr-un canal public
- Deci, un adversar poate intercepta toate mesajele transmise pe canalul de comunicatie, dar nu trebuie sa afle nimic despre cheia secreta
- Protocolele de schimb de chei sunt o primitiva fundamentala in criptografie

Schimbul de chei Diffie-Hellman

- Alice si Bob doresc sa stabileasca o cheie secreta comuna
- Alice genereaza un grup ciclic G de ordin q cu $|q| = n$ si g un generator al grupului

Alice alege $x \xleftarrow{R} \mathbb{Z}_q$ și calculează $h_1 := g^x$;

-
- Alice ii trimite lui Bob mesajul (G, g, q, h_1)
- Bob alege $y \xleftarrow{R} \mathbb{Z}_q$ și calculează $h_2 := g^y$;
- Bob ii trimite h_2 lui Alice si intoarce cheia $k_B := h_1^y$;
- Alice primeste h_2 si intoarce cheia $k_A = h_2^x$.

Corectitudinea protocolului

- Corectitudinea presupune $k_A = k_B$ ceea ce se verifica usor
- Bob calculeaza $k_B = h_1^y = (g^x)^y = g^{xy}$
- Alice calculeaza $k_A = h_2^x = (g^y)^x = g^{xy}$

Securitatea Diffie-Hellman

- Conditie **minimala** este ca DLP sa fie **dificila** in G . Aceasta nu este insa singura conditie pentru protectia impotriva unui atacator pasiv
- Cum poate un adversar sa determine cheia comuna daca poate sparge DLP? Ascuta mediu de comunicatie si preia mesajele h_1 si h_2 . Rezolva DLP pentru h_1 si determina x , apoi calculeaza cheile

Computational Diffie-Hellman(CDH)

- O conditie mai potrivita ar fi ca A sa nu poata determina cheia comuna $k_A = k_B$ chiar daca are acces la intreaga comunicatie
- **Problema de calculabilitate Diffie-Hellman (CDH):**
 - Fiind date grupul ciclic G , un generator g al sau si 2 elemente h_1, h_2 din G sa se determine: $CDH(h_1, h_2) = g^{\log_g h_1 \log_g h_2}$
- Pentru schimbul de chei Diffie-Hellman rezolvarea CDH inseamna ca adversarul determina cheile cunoscand h_1, h_2, G, g care sunt disponibile pe mediul de transmisiune nesecurizat

Decisional Diffie-Hellman(DDH)

- Nici conditia CDH nu este suficienta; chiar daca A nu poate determina cheia, acesta poate afla parti din ea
- O conditie si mai potrivita este ca pentru adversar, cheia $k_A = k_B$ sa fie **indistinctibila** fata de una aleatoare
- **Problema de decidabilitate Diffie-Hellman**

Definiție

Spunem că problema decizională Diffie-Hellman (DDH) este dificilă (relativ la $\mathbb{G}$), dacă pentru orice algoritm PPT $\mathcal{A}$ există o funcție neglijabilă negl așa încât:

$$|Pr[\mathcal{A}(\mathbb{G}, q, g, g^x, g^y, g^z) = 1] - Pr[\mathcal{A}(\mathbb{G}, q, g, g^x, g^y, g^{xy}) = 1]| \leq \text{negl}(n),$$

unde $x, y, z \xleftarrow{R} \mathbb{Z}_q$

Important de reținut:

- Prezumptii criptografice: CDH, DDH

Atacul Man-in-the-Middle(MITM)

Schimbul de chei DH este nesigur in cazul unui adversar activ care poate intercepta, modifica, elimina mesaje pe calea de comunicatie

- Alice genereaza un grup ciclic G , de ordin q cu $|q| = n$ si g un generator al grupului
- ► Alice alege $x \xleftarrow{R} \mathbb{Z}_q$ și calculează $h_1 := g^x$;
- Alice ii trimite lui Bob mesajul (G, g, q, h_1)
- Oscar intercepteaza mesajul si raspunde lui Alice in locul lui Bob:
alege $a \xleftarrow{R} \mathbb{Z}_q$ și calculează $h'_2 := g^a$;
- Oscar si Alice detin acum cheia comuna $k_A = g^{xa}$
- Oscar initiaza, in locul lui Alice, o noua sesiune cu Bob:
 $b \xleftarrow{R} \mathbb{Z}_q$ și calculează $h'_1 := g^b$;
- ► Bob alege $y \xleftarrow{R} \mathbb{Z}_q$ și calculează $h_2 := g^y$;
- Oscar si Bob detin acum cheia comuna $k_B = g^{yb}$

Atacul este posibil pentru ca Oscar ii poate **impersona** pe Alice si pe Bob. Oscar previne mesajele lui Alice sa ajunga la Bob, decriptandu-le folosind k_A si retrimittandu-le catre bob criptat cu k_B . Alice si Bob comunica fara sa stie de existenta lui Oscar.

10.4 Sistemul de criptare ElGamal

- Definim sistemul de criptare *ElGamal* pe baza ideii prezentate anterior;

1. Se generează $(\mathbb{G}, q, g)$, se alege $x \leftarrow^R \mathbb{Z}_q$ și se calculează $h = g^x$;
 - Cheia publică este: $(\mathbb{G}, q, g, h)$;
 - Cheia privată este x ;
2. **Enc:** dată o cheie publică $(\mathbb{G}, q, g, h)$ și un mesaj $m \in \mathbb{G}$, alege $y \leftarrow^R \mathbb{Z}_q$ și întoarce $c = (c_1, c_2) = (g^y, m \cdot h^y)$;
3. **Dec:** dată o cheie secretă $(\mathbb{G}, q, g, x)$ și un mesaj criptat $c = (c_1, c_2)$, întoarce $m = c_2 \cdot c_1^{-x}$.

Teorema - Securitatea ElGamal

Teoremă

Dacă problema decizională Diffie-Hellman (DDH) este dificilă în grupul $\mathbb{G}$, atunci schema de criptare ElGamal este CPA-sigură.

- Se poate vedea din securitatea schimbului de chei Diffie-Hellman
- In forma aceasta, sistemul ElGamal nu este CCA-sigur...pentru ca este maleabil
Insa poate fi modificat așa încât să fie CCA-sigur

Probleme, intrebari legate de ElGamal

Determinism - Este sistemul ElGamal determinist?

NU! Sistemul este nedeterminist, datorita alegerii aleatoare a lui y la fiecare criptare.

- Un același mesaj m se poate cripta diferit, pentru $y \neq y'$:

$$c = (c_1, c_2) = (g^y, m \cdot h^y)$$

$$c' = (c'_1, c'_2) = (g^{y'}, m \cdot h^{y'})$$

Dificultatea DLP - Ramane ElGamal sigur daca problema DLP este simpla?

- **Răspuns:** NU! Se determină x a.î. $h = g^x$, apoi se decriptează orice mesaj pentru că se cunoaște cheia secretă.

Proprietatea de homomorfism

- Fie m_1, m_2 2 texte clare și $c_1 = (c_{11}, c_{12}), c_2 = (c_{21}, c_{22})$ textele criptate corespunzătoare;
- Atunci:

$$c_1 \cdot c_2 = (c_{11} \cdot c_{21}, c_{12} \cdot c_{22}) = (g^{y_1} \cdot g^{y_2}, m_1 h^{y_1} \cdot m_2 h^{y_2})$$
- **Întrebare:** Dacă un adversar cunoaște c_1 și c_2 criptările lui m_1 , respectiv m_2 , ce poate spune despre $c_1 \cdot c_2$?
- **Răspuns:** $c_1 \cdot c_2$ este criptarea lui $m_1 \cdot m_2$ folosind $y = y_1 + y_2$:

$$c_1 \cdot c_2 = (g^{y_1+y_2}, m_1 m_2 h^{y_1+y_2})$$
- Un sistem de criptare care satisface

$$Dec_{sk}(c_1 \cdot c_2) = Dec_{sk}(c_1) \cdot Dec_{sk}(c_2)$$
 se numește sistem de criptare **homomorfic**.
 (homomorfismul este deseori o proprietate utilă în criptografie)

Utilizarea multiplă a parametrilor publici

- Este comun în practică pentru un administrator să fixeze parametrii publici (G, q, g) , apoi fiecare utilizator să își genereze doar cheia secretă x și să publice $h = g^x$;
- **Întrebare:** Este corect să se utilizeze de mai multe ori aceiași parametrii publici (G, q, g) ?
- **Răspuns:** Se consideră că DA. Cunoașterea parametrilor publici pare să nu conducă la rezolvarea DDH.
- **Atenție!** Acest lucru nu se întâmplă și la RSA, unde modulul NU trebuie utilizat de mai multe ori.

Curs 10.5 - Criptografia bazate pe Curbe Eliptice

Definiție

O curbă eliptică peste $\mathbb{Z}_p$, $p > 3$ prim, este mulțimea perechilor (x, y) cu $x, y \in \mathbb{Z}_p$ așa încât

$$y^2 = x^3 + Ax + B \mod p$$

împreună cu punctul la infinit $\mathcal{O}$ unde

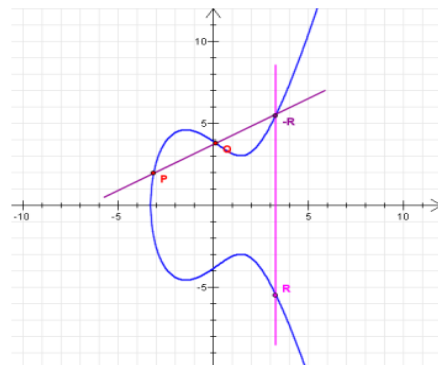
$A, B \in \mathbb{Z}_p$ sunt constante care respectă $4A^3 + 27B^2 \neq 0 \mod p$

- Vom nota cu $E(\mathbb{Z}_p)$ o curbă eliptică definită peste $\mathbb{Z}_p$

Exemplu adunare puncte $P + Q \rightarrow$

Geometric, suma a punctelor P și Q se obține trasând o linie prin cele două puncte și găsind cel de-al treilea punct R de intersecție al liniei cu E . m este panta dreptei care trece prin P și Q . Se poate arata că mulțimea de puncte împreună cu operația aditivă formează un grup Abelian. Există o teoremă de structură care exprimă condițiile în care grupul este ciclic.

O curbă eliptică peste spațiul numerelor reale $\mathbb{R}$
 $E(\mathbb{R}) : y^2 = x^3 - x + 1$



Curbe eliptice folosite în practică

- În practică, sunt cautate acele curbe eliptice pentru care ordinul grupului ciclic generat este prim
- Pentru criptografie, sunt de interes curbe eliptice de ordin mare
- O curbă eliptică peste $\mathbb{Z}_p$ are aproximativ p puncte.
- Teorema lui Hasse

$$p + 1 - 2\sqrt{p} \leq \text{card}(E(\mathbb{Z}_p)) \leq p + 1 + 2\sqrt{p}$$

- Există mai multe clase de curbe eliptice slabe d.p.d.v. criptografic care trebuie evitate, cum ar fi curbele eliptice peste $\mathbb{Z}_p$ cu $\text{card}(E(\mathbb{Z}_p)) = p$
- În practică se folosesc curbe eliptice standardizate

Curbe eliptice standardizate, folosite in practica, sigure si cu implementari eficiente

- P-256(sau secp256r1) - peste Z_p cu p pe 256 de biti
 - $p = 2^{256} - 2^{224} + 2^{192} - 2^{96} - 1$
 - Are ecuatia $y^2 = x^3 - 3x + B$
 - P-384(secp384r1) si P-521(secp521r1) sunt definite analog
- Curba 25519 este definita peste Z_p cu p pe 255 de biti de forma $p = 2^{255} - 19$ si permite o implementare eficienta. Grupul nu are ordin prim dar se poate lucra intr-un subgrup de ordin mare prim
- Secp256k1 este o curba eliptica de ordin prim peste Z_p cu p pe 256 de biti de forma $p = 2^{256} - 2^{232} - 2^{29} - 2^{28} - 2^{27} - 2^{26} - 2^{24} - 1$. Are ecuatia $y^2 = x^3 + 7 \bmod p$. Este **folosita in Bitcoin**.

ECDLP - Elliptic Curve Discrete Logarithm Problem

- Putem defini DLP in grupul punctelor unei curbe eliptice - ECDLP
- Fie E o curba eliptica peste Z_p , un punct P din Z_p de ordin n si Q un element din subgrupul ciclic generat de P $Q \in [P] = \{sP \mid 1 \leq s \leq n-1\}$
- Problema ECDLP necesita gasirea unui k asa incat $Q = kP$
- Notatie: $P + \dots + P$ de s ori $= sP$

Securitate ECDLP

- O consecinta a teoremei lui Hasse este ca daca avem nevoie de o curba eliptica cu 2^{160} de elemente, trebuie sa folosim un numar prim p pe aproximativ 160 de biti
- Deci daca folosim o curba eliptica $E(Z_p)$ cu p pe 160 de biti, un atac generic asupra ECDLP are 2^{80} complexitate timp
- Un nivel de securitate pe 80 de biti ofera securitate pe termen mediu
- In practica, curbele eliptice Z_p cu p pana la 256 de biti sunt folosite, cu un nivel de securitate pe 128 de biti

Criptografia pe Curbe Eliptice(Elliptic Curve Cryptography)

- Curbele eliptice sunt folosite pe larg si in standardele comerciale precum IPsec sau TLS
- ECC este adesea preferata in fata criptografiei cu cheie publica pentru sistemele incorporate precum disp. mobile **din motive de performanta**
- Implementarile pentru ECC sunt considerabil mai mici si mai rapide decat pentru RSA
- ECC cu chei pe 160-250 de biti ofera cam acelasi nivel de securitate precum RSA sau sistemele bazate pe DLP cu chei pe 1024-3072 de biti

Security level

- Un algorithm are nivelul de securitate "security level" pe n biti daca cel mai bun atac necesita 2^n pasi

Algorithm Family	Cryptosystems	Security Level (bit)			
		80	128	192	256
Integer factorization	RSA	1024 bit	3072 bit	7680 bit	15360 bit
Discrete logarithm	DH, DSA, Elgamal	1024 bit	3072 bit	7680 bit	15360 bit
Elliptic curves	ECDH, ECDSA	160 bit	256 bit	384 bit	512 bit
Symmetric-key	AES, 3DES	80 bit	128 bit	192 bit	256 bit

Important de retinut

- ECDLP este dificila
- Curbele eliptice ofera un suport bun pentru criptografie

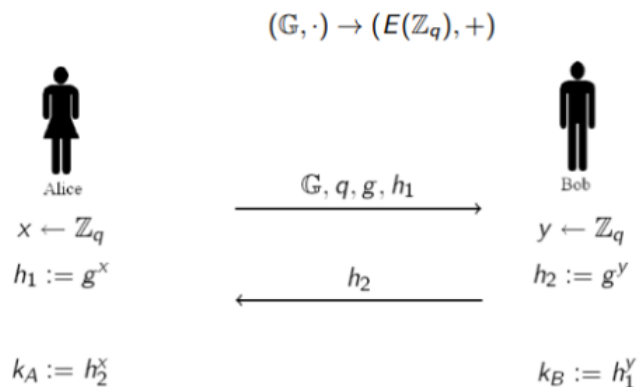
Curs 10.6 Schimbul de chei Diffie-Hellman si ElGamal pe Curbe Eliptice

- Am studiat schimbul de chei Diffie-Hellman peste un grup ciclic G de ordin q
- Transpunem constructia pe curbe eliptice

Diffie-Hellman pe curbe eliptice

- Alice si bob doresc sa stabileasca o cheie secreta comuna
- Alice genereaza o curba eliptica $E(\mathbb{Z}_q)$ si un punct P pe curba (generator)

- Alice alege $x \xleftarrow{R} \mathbb{Z}_q$ și calculează $H_1 := xP$;
- Alice ii trimite lui Bob mesajul $(E(\mathbb{Z}_q), P, H_1)$
- Bob alege $y \xleftarrow{R} \mathbb{Z}_q$ și calculează $H_2 := yP$;
- Bob ii trimite H_2 lui Alice si intoarce cheia $k_B = yH_1$
- Alice primeste H_2 si intoarce cheia $k_A = xH_2$



Corectitudine

- Presupune $k_A = k_B$, ceea ce se verifica usor
- Bob calculeaza $k_B = yH_1 = y(xP) = (xy)P$
- Alice calculeaza $k_A = xH_2 = x(yP) = (xy)P$

Securitate

- O conditie **minimala** pentru securitate este ca ECDLP sa fie dificila in G . Aceasta nu este insa singura conditie pentru a proteja protocolul de un atacator pasiv
- Cum poate un adversar pasiv sa determine cheia daca poate sparge ECDLP? Preia mesajele H_1 si H_2 . Rezolva ECDLP pentru H_1 si determina x , apoi calculeaza $k_A = k_B = xH_2$

ECGDH - Elliptic Curve Computational Diffie-Hellman

- O conditie mai potrivita ar fi ca adversarul sa nu poata determina cheia comuna $k_A = k_B$ chiar daca are acces la intreaga comunicare
- ECGDH - Fiind date curba eliptica $E(\mathbb{Z}_q)$, un punct P pe curba si 2 alte puncte H_1, H_2 de pe curba sa se determine

$$ECGDH(H_1, H_2) = (ECDLP(P, H_1)ECDLP(P, H_2))P$$

- Rezolvarea ECCDH inseamna ca adversarul determina cheia cunoscand $H1, H2, E(\mathbb{Z}_q), P$ care sunt disponibile pe mediul de transmisiune nesecurizat

ECDDH - Elliptic Curve Decisional Diffie-Hellman

- Nici ECCDH nu este suficienta, chiar daca adversarul nu poate determina cheia acesta poate spre ex. sa determine parti din ea
- O conditie mai potrivita este ca $k_A = k_B$ sa fie **indistinctibila** fata de o valoare aleatoare
- **Definitie ECCDH**

Definiție

Spunem că problema decizională Diffie-Hellman (ECDDH) este dificilă (relativ la curba eliptică $E(\mathbb{Z}_q)$), dacă pentru orice algoritm PPT $\mathcal{A}$ există o funcție neglijabilă negl așa încât:

$$|Pr[\mathcal{A}(E(\mathbb{Z}_q), P, xP, yP, zP) = 1] - Pr[\mathcal{A}(E(\mathbb{Z}_q), P, xP, yP, xyP) = 1]| \leq \text{negl}(n), \text{ unde } x, y, z \xleftarrow{R} \mathbb{Z}_q$$

Atacul Man-in-the-Middle pe Diffie-Hellman pe curbe eliptice

- Alice genereaza o curba eliptica $E(\mathbb{Z}_q)$ si P un punct pe curba. Alege x din $\mathbb{Z}_q$ si calculeaza $H1 = xP$. Alice ii trimite lui Bob $(E(\mathbb{Z}_q), P, H1)$
- Oscar intercepteaza mesajul si raspunde lui Alice in locul lui Bob - alege a din $\mathbb{Z}_q$ si calculeaza $H2 = aP$. Oscar si Alice au acum cheia comuna $k_A = axP$
- Oscar initiaza in locul lui Alice o sesiune cu Bob. Alege b de pe $\mathbb{Z}_q$ si calculeaza $H'1 = bP$, pe care le trimite lui Bob
- Bob alege y din $\mathbb{Z}_q$ si calculeaza $H2 = yP$
- Oscar si bob detin acum cheia comuna $k_B = ybP$

Atacul este posibil pentru ca Oscar ii poate **impersona** pe Alice si pe Bob. Oscar previne mesajele lui Alice sa ajunga la Bob, decriptandu-le folosind k_A si retransmitandu-le catre bob criptat cu k_B . Alice si Bob comunica fara sa stie de existenta lui Oscar.

ElGamal pe Curbe Eliptice

Am studiat sistemul de criptare ElGamal peste un grup ciclic G de ordin q . Transpunem constructia pe

curbe eliptice: $(\mathbb{G}, \cdot) \rightarrow (E(\mathbb{Z}_q), +)$

Sistemul de criptare

1. Se generează $E(\mathbb{Z}_q)$ o curbă eliptică și P un punct pe curbă (generator), se alege $z \xleftarrow{R} \mathbb{Z}_q$ și se calculează $H = xP$;
 - Cheia publică este: $(E(\mathbb{Z}_q), P, H)$;
 - Cheia privată este $(E(\mathbb{Z}_q), P, x)$;
2. **Enc:** dată o cheie publică $(E(\mathbb{Z}_q), P, H)$ și un mesaj $M \in E(\mathbb{Z}_q)$, alege $y \xleftarrow{R} \mathbb{Z}_q$ și întoarce $C = (C_1, C_2) = (yP, M + yH)$;
3. **Dec:** dată o cheie secretă $(E(\mathbb{Z}_q), P, x)$ și un mesaj criptat $C = (C_1, C_2)$, întoarce $M = C_2 + x(-C_1)$.

Securitate

- Sistemul transpus pe curbe eliptice pastreaza proprietatile sistemului initial

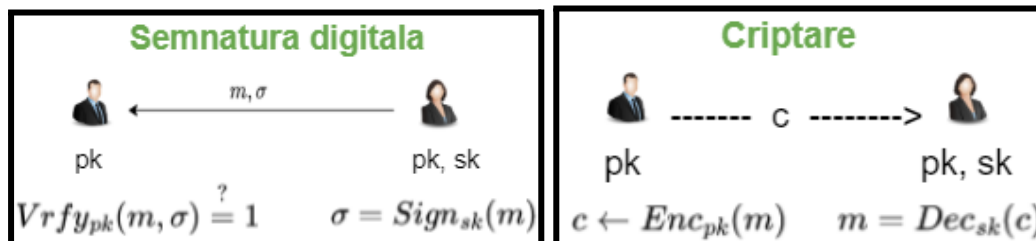
- Deci curba eliptica trebuie aleasa a.i. **ECDLP** si **ECDDH** sa fie dificile si sistemul ramane determinist si homomorfic

Important de retinut

- Modalitatea de trecere de la o constructie peste $(\mathbb{Z}_q, *)$ la $(E(\mathbb{Z}_q), +)$
- Prezumptii criptografice: ECDDH, ECDDH
- Schimbul de chei Diffie-Hellman pe curbe eliptice pastreaza proprietatile schimbului de chei Diffie-Hellman definit peste G grup ciclic de ordin q

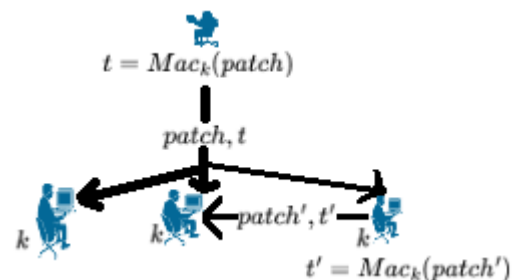
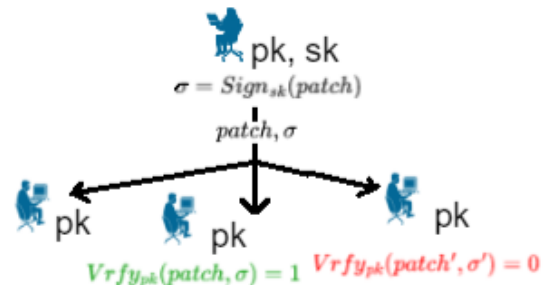
Curs 11: Semnaturi digitale si PKI

- **Schemele de semnatura digitala** repr echivalentul MAC-urilor in criptografia cu cheie publica, desi exista cateva dif intre ele;
- O schema de semnatura digitala ii permite unui semnatar S care a stabilit o cheie publica pk sa semneze un mesaj in asa fel incat oricine care cunoaste cheia pk poate verif originea mesajului (ca este S) si integritatea lui;



Aplicatii ale semnaturilor digitale

- O comp. soft. vrea sa transmita patch-uri de software intr-o maniera autentificata a.i. $\forall$ client sa poata recunoaste daca un patch e autentic
 - In schimb, o persoana malitioasa nu poate pacali un client sa accepte un patch care nu a fost realizat de compania respectiva
 - Pt aceasta compania genereaza o cheie publica pk impreuna cu o cheie secreta sk si da pk chlientilor sai, pastrand sk
 - Atunci cand lanseaza un patch de soft. $patch$, comp. calc. o semnatura digit. σ pt $patch$ folosind sk si trimite fiecarui client perechea $(patch, \sigma)$;
 - Fiecare client stabileste autenticitatea lui $patch$ verificand daca σ este o semnatura legitima pt $patch$ cu privire la cheia publica pk .
 - Deci compania foloseste aceeasi cheie publica pt toti clientii si calculeaza o singura semnatura pe care o trimite tuturor
 - Putem inlocui semnatura digitala cu un MAC?
- In aceasta situatie, oricare dintre clienti poate emite un tag valid pt un alt patch decat cel original si chiar il poate trimite celorlalti clienti



Avantaje semnaturi digitale fata de MAC-uri

- MAC-urile si schemele de semnatura digitala sunt folosite pt asigurarea integritatii (autenticitatii) mesajelor cu urmatoarele **diferente**:

- Schemele de semnatura digitala sunt *public verificabile*...
- ...ceea ce inseamna ca semnaturile digitale sunt *transferabile* - o terta parte poate verifica legitimitatea unei semnaturi si poate face o copie pt a convinge pe altcineva ca aceea este o semnatura valida pentru m;
- Schemele de semnatura digitala au proprietatea de *non-repudiare* - un semnatar nu poate nega faptul ca a semnat un mesaj;
- MAC-urile au avantajul ca sunt cam de 2-3 ori mai eficiente (mai rapide) decat schemele de semnatura digitala.

Semnaturi digitale - Definitie

Def: O semnatura digitala definita peste $(\mathcal{K}, \mathcal{M}, \mathcal{S})$ este formata din trei algoritmi polinomiali (Gen, Sign, Vrfy) unde:

1. Gen: alg. de generare a perechii de cheie publica si privata (pk, sk)
2. Sign : $\mathcal{K} \times \mathcal{M} \rightarrow \mathcal{S}$ alg. de generare a semnaturilor $\sigma \leftarrow \text{Sign}_{sk}(m)$;
3. Vrfy : $\mathcal{K} \times \mathcal{M} \times \mathcal{S} \rightarrow \{0, 1\}$ alg. de verificare ce intoarce un bit $b = \text{Vrfy}_{pk}(m, \sigma)$ cu semnificatia ca: $b = 1 \Rightarrow \text{valid} \mid b = 0 \Rightarrow \text{invalid}$

a.i. $\forall m \in \mathcal{M}, k \in \mathcal{K}, \text{Vrfy}_{pk}(m, \text{Sign}_{sk}(m)) = 1$

Securitate semnatura digitala - formalizare

- Formal, ii dam adversarului acces la un oracol $\text{Sign}_{sk}(\cdot)$;
- Adversarul poate trimite orice mesaj m dorit catre oracol si primeste inapoi o semnatura corespunzatoare $\sigma \leftarrow \text{Sign}_{sk}(m)$;
- Consideram ca securitatea este impactata daca adversarul este capabil sa produca un mesaj m impreuna cu o semnatura σ a.i.:

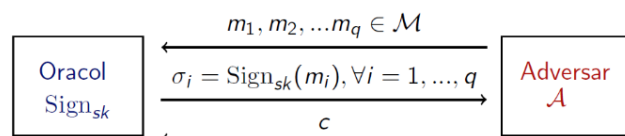
1. σ este o semnatura valida pt mesajul m: $\text{Vrfy}_{pk}(m, \sigma) = 1$;
2. Adversarul nu a solicitat anterior (de la oracol) o semnatura pt m

- Despre o semnatura care satisface nivelul de securitate de mai sus spunem ca *nu poate fi falsificata printr-un atac cu mesaj ales*;
- Aceasta inseamna ca un adversar nu este capabil sa falsifice o semnatura valida pt niciun mesaj...
- ...desi poate obtine semnaturi pt orice mesaj ales de el, chiar *adaptiv* in timpul atacului.

- Pt a da def formala, definim mai intai un experiment pt o semnatura $\pi = (\text{Sign}, \text{Vrfy})$, in care consideram un adversar A si parametrul de securitate n;

Experimentul $\text{Sign}_{A,\pi}^{\text{forge}}(n)$

- Adversarul intoarce o pereche de mesaj, semnatura (m, σ)



- Outputul experimentului este $1 \Leftrightarrow$

(1) $\text{Vrfy}_{pk}(m, \sigma) = 1$ și (2) $m \notin \{m_1, \dots, m_q\}$;

- Daca $\text{Sign}_{A,\pi}^{\text{forge}}(n) = 1$, spunem ca A a efectuat exp-mentul cu succes

Securitate semnaturi digitale

Def: O semnatura $\pi = (\text{Gen}, \text{Sign}, \text{Vrfy})$ este sigura (nu poate fi falsificata printr-un atac cu mesaj ales) daca pt orice adversar polinomial A exista o functie neglijabila negl a.i.

$$\Pr[\text{Sign}_{A,\pi}^{\text{forge}}(n) = 1] \leq \text{negl}(n).$$

Atacuri prin replicare

- Un atacator poate prelua o pereche de mesaj si semnatura digitala si o retrimite unui destinatar
- la fel ca in criptografia simetrica, definitia semnatuurilor digitale nu protejeaza impotriva acestui tip de atac
- in contextul exemplului cu patch-ul de software, acest atac este problematic

Constructie de scheme de semnatura digitala

- Paradigma "hash-and-sign" este sigura: inainte de semnare, mesajul trece printr-o fct hash; varianta aceasta se foloseste pe larg in practica:

$$\triangleright \sigma \leftarrow \text{Sign}_{sk}(H(m)); \text{Vrfy}_{pk}(H(m), \sigma) \stackrel{?}{=} 1$$

- constructia este sigura atata timp cat H este rezistent la coliziuni
- e avantajoasa pt ca ofera functionalit. unei semnat. digitale (criptografie cu cheie publica) la costul unei operatii din criptografia cu cheie secreta
- folosita pe larg in practica

(Vezi prezumptia RSA)

Semnatura digitala bazata pe RSA

- Gen: genereaza N, e, d si pk = (N, e) iar sk = d
 - Sign(sk, m): semneaza mesajul m folosind cheia sk = d astfel: $\sigma = m^d \bmod N$
 - Vrfy(pk, m, σ): semnatura este valida $\Leftrightarrow m \stackrel{?}{=} \sigma^e \bmod N$
- Aceasta varianta de semnatura nu este sigura, se cunosc mai multe atacuri pt ea

Un atac asupra semnaturii bazate pe RSA

- scop adversar: falsificarea semnaturii mesajului $m \in \mathbb{Z}_N^*$ pt pk = (N, e)
 - actiune adversar: alege $m_1, m_2 \in \mathbb{Z}_N^*$ a.i. $m = m_1 \cdot m_2 \bmod N$
 - obtine semnatuurile σ_1 și σ_2 pentru mesajele m_1, m_2
 - întoarce $\sigma = \sigma_1 \cdot \sigma_2 \bmod N$ ca semnătură validă pentru m
 - aceasta este o semnatura valida pt ca
- $$\sigma^e = (\sigma_1 \cdot \sigma_2)^e = (m_1^d \cdot m_2^d)^e = m_1^{ed} \cdot m_2^{ed} = m_1 \cdot m_2 = m \bmod N$$

Varianta RSA-FDH (full-domain hash)

- Gen: genereaza N, e, d si pk = (N, E) iar sk = d
- Sign: semenaza m folosind cheia sk = d astfel: $\sigma = H(m)^d \bmod N$
- Vrfy(pk, m, σ): semnatura este valida $\Leftrightarrow H(m) \stackrel{?}{=} \sigma^e \bmod N$
- Se poate verifica usor daca atacul precedent nu functioneaza:

$$H(m_1) \cdot H(m_1) = \sigma_1^e \cdot \sigma_2^e = (\sigma_1 \cdot \sigma_2)^e \neq H(m_1 \cdot m_2)$$

Aceasta var. de semnatura este sigura daca H indeplineste doua cond.:

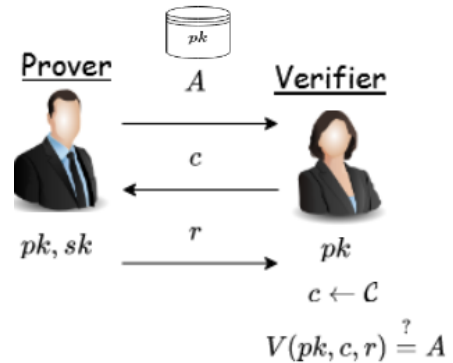
- H este rezistent la coliziuni
- $H: \{0,1\}^* \rightarrow \mathbb{Z}_N^*$

Scheme de identificare - identification schemes

- sunt protocoale interactive care permit unei parti (Prover) sa isi demonstreze identitatea in fata unei alte parti (Verifier)
- sunt f. imp. ca building block pt semn. digit. (dar au aplicabilit. redusa)
- in continuare, abordare informala:

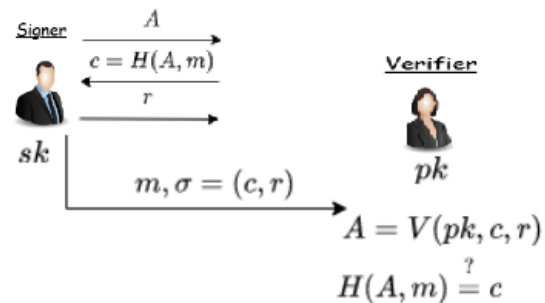
Scheme de identificare

- Securitate
 - fata de adversarii *pasivi* - chiar daca are acces la mesajele trimise in mai multe executii ale protocolului, un adversar nu il poate convinge pe Verifier sa accepte
- Principala aplicatie
 - identificarea persoanelor prezente fizic; de exp., deschiderea unei usi securizate pe baza unei cartele de acces
 - nu este potrivita pt autentificarea la distanta (pe internet)



Transformarea schemelor de identificare in sematuri digitale

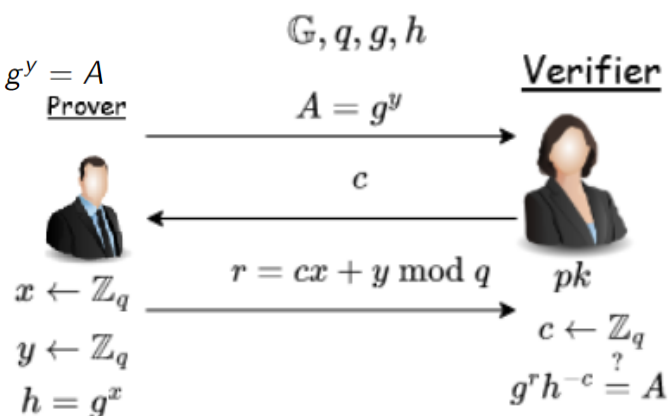
- Pt a semna, Proverul executa singur protocolul generand challenge-ul pe baza unei fct hash - foloseste transformarea *Fiat-Shamir*



Schema de identificare Schnorr

- In schema de identificare de mai jos G este un grup ciclic de ordin q si generator g , $sk = x$ si $pk = (G, q, g, h)$ unde $h = g^x$
- Se poate verifica usor ca:

$$g^r h^{-c} = g^{cx+y} h^{-c} = (g^x)^c g^y h^{-c} = g^y = A$$



Schema de identificare Schnorr - securitate

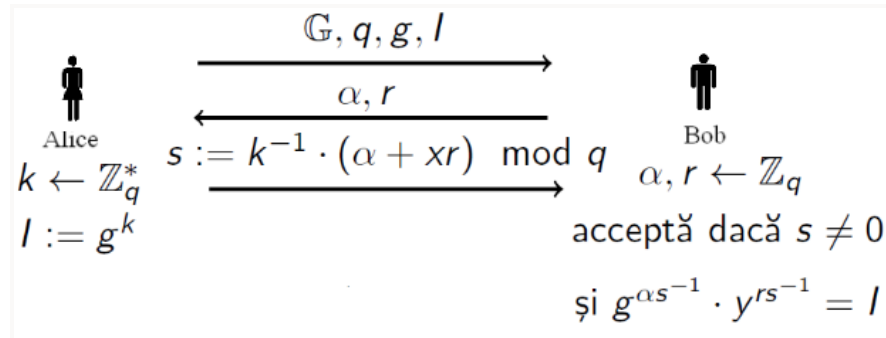
- Daca problema logaritmului discret este dificila, atunci schema de identificare Schnorr este sigura impotriva atacurilor pasive
- Daca problema logaritmului este dificila si H este modelata ca o fct aleatoare, atunci semnatura Schnorr (obt. din schema de identif. Schnorr prezentata anterior, prin aplicarea transformatei Fiat-Shamir) este sigura

Alte scheme de semnatura digitala

- Un alt exp. folosit in practica este Digital Signature Algorithm (DSA) bazat pe pb. logaritmului discret, dar si ECDSA (varianta DSA bazata pe curbe eliptice), ambele fiind incluse in DSS (Digital Signature Standard).
- Atat DSA cat si ECDSA se bazeaza pe PLD in diferite clase de grupuri

DSA/ECDSA

Sunt construite pe baza schemei de identificare de mai jos unde G este un grup ciclic de ordin q si generator g , $sk = x$ si $pk = (G, q, g, y)$ unde $y = g^x$,



- Se poate verif. usor ca schema e corecta: $s = 0$ doar daca $\alpha = -xr \bmod q$ ceea ce se intampla cu probabilitate neglijabila

- Considerand $s \neq 0$, $s^{-1} \bmod q$ există și

$$g^{\alpha s^{-1}} \cdot y^{r s^{-1}} = g^{\alpha s^{-1}} \cdot g^{x r s^{-1}} = g^{(\alpha + xr) s^{-1}} = g^{(\alpha + xr) \cdot k \cdot (\alpha + xr)^{-1}} = I$$

- Schema anterioara este sigura daca PLD este dificila in grupul G .

- Schemele de semnatura DSA/ECDSA se obtin prin transformarea schemei de sus intr-una non-interactiva.

- Modificarile pentru a face schema non-interactiva sunt urmatoarele:

- Notar $\alpha = H(m)$, unde H este o fct hash criptografica
- $r = F(I)$ pt o fct specifica $F : \mathbb{G} \rightarrow \mathbb{Z}_q$
- Varianta non-interactiva a schemei esta mai jos

* Gen: genereaza G un gr ciclic de ord q si un generator g , alege uniform $x \in \mathbb{Z}_q$ și $y = g^x$. Cheia publica este $pk = (G, q, g, y)$, iar cheia secreta $sk = x$. Se alege fct $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ și $F : \mathbb{G} \rightarrow \mathbb{Z}_q$.

* Sign(sk, m): alege uniform $k \in \mathbb{Z}_q^*$ și $r = F(g^k)$. Calculeaza $s := k^{-1} \cdot (H(m) + xr) \bmod q$. Daca $s=0$ sau $r=0$ se reincepe cu o noua alegere a lui k . Semnatura rezultata este (r, s) .

* Vrfy($pk, m, (r, s)$): semnat. e valida $\Leftrightarrow r \stackrel{?}{=} F(g^{H(m) \cdot s^{-1}} y^{r \cdot s^{-1}})$

DSA/ECDSA - Securitate

- Securitatea semnaturii DSA/ECDSA se bazeaza pe problema logaritmului discret si pe faptul ca F si G sunt alese corespunzator.

- E f important ca la generarea semnat. sa fie ales aleator k (sa nu fie predictibil). Altfel, cheia secreta x se poate afla (in ecuatie $s = k^{-1} \cdot (H(m) + xr) \bmod q$, singura necunoscuta este x).

- Also, folosirea ac. k pt a genera 2 semnat. dif. $\Rightarrow$ gasirea cheii secrete.

Certificate si PKI

- O pb. a criptografiei cu cheie publica o repr. distribuirea cheilor publice;
- Se rezolva tot cu criptografia cu cheie publica: e suficient sa distribuim o singura cheie publica in mod sigur...
- Dupa ea poate fi folosita pt a distribui sigur oricat de multe chei publice

- Ideea: folosirea unui *certificat digital* care e o semnatura care ataseaza unei entitati o anume cheie publica;

- De exp. daca Charlie are cheia generata (pk_C, sk_C) si Bob are cheia (pk_B, sk_B), iar Charlie stie atunci el poate calcula semnatura urmat. pe care i-o da lui Bob:

$$cert_{C \rightarrow B} = \text{Sign}_{sk_C}("Cheia\ lui\ Bob\ este\ pk_B")$$

- Aceasta semnat. e un *certificat* emis de Charlie pt Bob

- Cand Bob vrea sa comunice cu Alice ii trimite intai cheia publica impreuna cu certif. $cert_{C \rightarrow B}$ a carui validitate in rap. cu pk_C Alice o verific.;

- Raman cateva pb: cum afla Alice pk_C , cum poate fi Charlie sigur ca e cheia pub. a lui Bob, cum decide Alice daca are incredere in Charlie

- Toate acestea sunt specif. intr-o *infrastruct. cu chei publice* (PKI) care permite distribuirea la scara larga a cheilor publice;

- Exista mai multe modele dif. de PKI, dupa cum vom vedea in cont.;

PKI cu o singura autoritate de certificare

- Aici exista o singura autoritate de certificare (CA) in care toate lumea are incredere si care emite certificate pt toate cheile publice;

- CA e o companie sau agentie guvern. sau un depart. dintr-o organiz.

- Cn. folos. serviciile CA trb sa obt. o copie legitima a cheii ei pub.

- Cheia pk_{CA} se obt. chiar prin mij. fizice; acest pas se face doar o data

PKI cu mai multe autoritati de certificare

- Modelul cu o singura CA nu e practic;

- In modelul cu multiple CA, daca Bob vrea sa obt. un certif. pt cheia lui pub. poate apela la ce CA vrea, iar Alice, care primeste un certif. sau mai multe poate alege in care CA sa aiba incredere;

- De exp., browserele web vin preconfig. cu un nr de chei pub. ale unor CA stabilite ca toate fiind de incredere in mod egal;

- Utilizatorul poate modif. aceasta config. a.i. sa accepte doar certificate de la CA-uri in care el are incredere

Delegare si lanturi de certificate

- Charlie e un CA care emite certif., inclusiv pt Bob

- Daca pk_B e o cheie pub. pt semnatura, atunci Bob poate emite certif. pt alte pers.; un certif. pt Alice are forma $cert_{B \rightarrow A} = \text{Sign}_{sk_B}("Cheia\ lui\ Alice\ este\ pk_A")$

- Atunci cand vb. cu Dan, Alice ii trimite $pk_A, cert_{B \rightarrow A}, pk_B, cert_{C \rightarrow B}$

- De fapt, $cert_{C \rightarrow B}$ contine, in afara de pk_B , si afirm. "Bob e de incredere pt a emite certificate"; asa, Charlie il deleaga pe Bob sa emita certificate

- Totul se poate organiza ca o ierarhie unde exista un CA "radacina" pe primul nivel si n CA-uri pe al doilea nivel

Modelul "web of trust"

- Aici oricine poate emite certif. pt oricine altcuv. si fiecare utilizat. decide cat de multa incredere poate acorda certificatelor emise de alti utilizatori.

- De exp., daca Alice are cheile pub. pk_1, pk_2, pk_3 coresp. lui C1, C2, C3 iar Bob, care vrea sa comunice cu Alice, are certificatele $cert_{C1 \rightarrow B}, cert_{C2 \rightarrow B}, cert_{C3 \rightarrow B}$ pe care i le trimite lui Alice

- Alice nu are pk_4 si nu poate verific. $cert_{C4 \rightarrow B}$; deci ca sa accepte pk_B

Alice trebuie sa decida cat de multa incredere are in C1 si C3

- Modelul e atractiv pt ca nu necesita incredere intr-o autoritate centrala;

Invalidarea certificatelor

- Atunci cand un angajat paraseste compania sau isi pierde cheia secreta, certificatul lui trb invalidat

- Exista mai multe metode de invalidare intre care:

- **Expirarea.** Se poate include data de exp. ca parte a unui certif., care trb. verific. impreuna cu validitatea semnaturii
- **Revocarea.** CA-ul poate, in mod explicit, revoca un certif. imediat ce nu mai poate fi folosit
- Aceasta se poate realiz. prin includerea unui nr. serial in certif.; la sf. unei zile CA genereaza o lista de certif. revocate (care contine nr. seriale) pe care o distribuie sau publica

Curs 12: Criptografia post-cuantica

- In criptografia de pana acum am vb de un adversar PPT care ruleaza in timp polinomial pe un *calculator conventional (clasic)*. In eval. securit. primitivelor criptografie am consider. numai *atacuri clasice*.
- Nu am avut in vedere calculatoarele cuantice - care se bazeaza pe principiile mecanicii cuantice si impactul lor asupra securitatii
- Algo. cuantici pot fi, in anum. cazuri, mult mai rapizi decat cei clasici + un impact zdrobitor asupra securitatii primitivelor criptografice studiate.
- Practic, nu exista un calculator cuantic generic pe scara larga, iar costurile pt constr. lui ar fi urias
- Totusi, un calculator cuantic dedicat care sa atace sistemele de criptare actuale ar putea aprea in decurs de cativa ani sau zeci de ani.
- Odata ce un astfel de calculator cuantic care sa poata fi folosit in practica devine disponibil, toti algoritmi cu cheie publica folositi in prezent dar si protocoalele asociate devin vulnerabile
- Deci toate emailurile, info. despre cardurile cu care facem plati online, semnaturi digit., tranzact. online, datele sensibile, info. clasificate ale agentilor de securitate si cele guvernamentale vor fi in pericol

Criptografia post-cuantica vs criptografia cuantica

Criptografia cuantică

- ▶ implementări folosind calculatoare cuantice, fenomene mecanice cuantice și canale de comunicare cuantice
- ▶ dificil de implementat la scara largă
- ▶ în unele cazuri este sigură necondiționat (nu se bazează pe ipoteze matematice)

Criptografia post-cuantică

- ▶ implementări folosind calculatoare clasice
- ▶ este sigură chiar și în fața unui adversar care are acces la un calculator cuantic
- ▶ se bazează pe probleme matematice dificile computațional chiar și pentru algoritmi cuantici

Criptografia simetrica post-cuantica

- Impactul calculatoarelor cunatice asupra criptografiei simetrice e minor
- Consider. urm. **pb. abstracta**: Se da: fct $f : D \rightarrow \{0,1\}$ cu acces de tip oracol (fct poate fi interogata pe $\forall$ input si se primeste outputul coresp.)
Se cere: sa se gaseasca x a.i. $f(x) = 1$
- Daca exista un singur x cu $f(x) = 1$ atunci orice algo. clasic necesita $O(\|D\|)$ evaluari ale fct f - coresp. unui atac prin forta bruta
- **1996 - algo. cuantic al lui Grover**: gaseste x folosind $O(\|D^{1/2}\|)$ evaluari ale fct f . Algoritmul este optim si nu poate fi imbunatatit

Criptografia simetrica post-cuantica - sisteme bloc

- Trecem in revista impactul algoritmului asupra sistemelor de criptare simetrice
- Consider. un sist. de cript. bloc $F : \{0, 1\}^n \times \{0, 1\}^l \rightarrow \{0, 1\}^l$ cu cheia k pe n biti pt care cel mai bun atac (de gasire a cheii) e forta bruta.
- Un astfel de atac clasic necesita timp 2^n
- Pt securitate, alegem cheia k de lungime n biti a.i. timpul pt atac 2^n sa fie nepractic.
- Algo. lui Grover insa: atacatorul gaseste cheia in timp $2^{(n/2)}$
- Pt acelasi nivel de securitate (precum in cazul clasic), alegem cheia k de lung **dubla** fata de cazul clasic.

Criptografia simetrica post-cuantica - functii hash

- Fie pb. gasirii de coliziuni pt o fct hash $H : \{0, 1\}^m \rightarrow \{0, 1\}^n$ cu $m > n$
- In cazul **clasic**, "atacul nasterilor" necesita $O(2^{(n/2)})$ eval. ale fct. H
- Aceasta inseamna ca pt a asigura rezistenta la coliziuni fata de un atac in timp 2^t , trb sa alegem fct hash cu **outputul pe $2t$ biti**.
- Insa in cazul **cuantic**, un atac pt gasirea coliziunilor necesita $O(2^{(n/3)})$ evaluari ale fct H .
- Deci pt a asigura rezistenta la coliz. fata de un atac in timp 2^t trb sa alegem fct hash cu **outputul pe $3t$ biti**.

Algoritmul lui Shor si impactul lui asupra criptografiei asimetrice

- Pana acum am vazut algoritmi cuantici care ofera o imbunatatire de *ordin polinomial* in comp. cu cei mai buni algoritmi clasici pt aceeasi pb.
- Acestia impun doar cresterea dimensiunilor chielor fara a necesita alte schimbari majore
- In continuare vom vedea un alg. care ofera o imbunatatire de *ordin exponential* - **alg. cuantici polinomiali pentru problema factorizarii si problema logaritmului discret**

Algoritmul lui Shor si impactul lui asupra criptografiei asimetrice

- Incepem cu o pb abstracta:
 - Se da: o fct $f : \mathbb{G} \rightarrow R$, cu $\mathbb{G}$ un grup comutativ
 - Pp ca f periodica: **există $\alpha \in \mathbb{G}$ - perioadă** cu $f(x) = f(x + \alpha)$
 - Se cere: gasirea unei perioade avand acces de tip oracol la fct f
- Nu se cunosc algo. clasici eficienti pt rezolvarea acestei pb.

Algoritmul lui Shor si impactul lui asupra criptografiei asimetrice

- 1994 - Shor - rezultat uimitor: algoritm cuantic polinomial care rezolva problema pt anumite grupuri
- Este un instrument puternic care poate fi folosit pt a factoriza si a calcula logaritmi discreti.
- Trb doar aleasa cu grija fct a carei perioada $\mathbb{G}$ ne da solutia cautata

Algoritmul lui Shor si impactul lui asupra factorizarii

- Fie pb. factorizarii: fie N un produs de doua nr prime. Pt orice $x \in \mathbb{Z}_N^*$

definim fct $f_{x,N} : \mathbb{Z} \rightarrow \mathbb{Z}_N^*$, $f_{x,N}(r) = x^r \bmod N$

- Principala observatie este ca fct are perioada $\phi(N)$ deoarece

$$f_{x,N}(r + \phi(N)) = x^{r + \phi(N)} \bmod N = x^r \cdot x^{\phi(N)} \bmod N = x^r \bmod N$$

- Pt orice x ales de noi, putem calcula, in timp polinomial, cu algo. lui Shor, o perioada a fct $f_{x,N}$ adica un $k \neq 0$ cu $x^k = 1 \bmod N$.

Aceasta imediat permite factorizarea lui N in timp polinomial.

Algoritmul lui Shor si impactul lui asupra criptografiei asimetrice

- Algo. lui Shor poate fi folos. si pt rez. pb. log. discret in timp polinomial
- Avand in vedere ca toate sist. de criptare cu cheie pub. se bazeaza pe pb. factorizarii sau pb. log. discret, concluzionam ca "Toate sist. de criptare cu cheie pub. prezentate la curs pot fi atacate in timp polinomial cu ajutorul unui calculator cuantic"

Sisteme de criptare cu cheie publica post-cuantice

- Pb. factorizarii si pb. log. discret devin "usoare" pt un calculator cuantic
- Avem nevoie de pb. matematice dificile computational chiar si pt calculatoare cuantice, dar care ruleaza pe calculatoare clasice
- Dif. fata de cazul clasic este ca pb. considerate pt. criptografia post-cuantica sunt mai recente si nu au fost studiate la fel de mult ca pb. factorizarii sau pb. log. discret
- In cont. vom prezenta o pb. care a primit multa atentie si care este considerata dificila chiar si pt calc. cuantice. Aratam apoi cum se poate construi un sist. de criptare cu cheie pub. bazat pe dificultatea acelei pb.

Problema LWE - Learning With Errors

- 2005, de Oded Regev

- Preliminarii:

- q nr prim. Vom nota cu $\mathbb{Z}_q$ mult. $\{-(q-1)/2, \dots, 0, \dots, [q/2]\}$ (spre deosebire de $\{0, \dots, q\}$) unde $[x]$ este cel mai mare intreg mai mic sau egal cu x .
- spunem ca un elem. al lui $\mathbb{Z}_q$ este "mic" daca este "aproape" de 0
- Pb cere gasirea lui $\mathbf{s} \in \mathbb{Z}_q^n$ fiind data o secv. de ec. lin. "aprox." in $\mathbf{s}$.
- Exp:

$$\begin{array}{l} 12s_1 + 10s_2 + 5s_3 + 2s_4 \approx 8 \text{ mod } 17 \\ 3s_1 + 7s_2 + 9s_3 + s_4 \approx 4 \text{ mod } 17 \\ 16s_1 + 2s_2 + 8s_3 + 7s_4 \approx 3 \text{ mod } 17 \end{array} \quad \begin{array}{l} 12s_1 + 10s_2 + 5s_3 + 2s_4 + 1 = 8 \text{ mod } 17 \\ 3s_1 + 7s_2 + 9s_3 + s_4 - 1 = 4 \text{ mod } 17 \\ 16s_1 + 2s_2 + 8s_3 + 7s_4 + 2 = 3 \text{ mod } 17 \end{array}$$

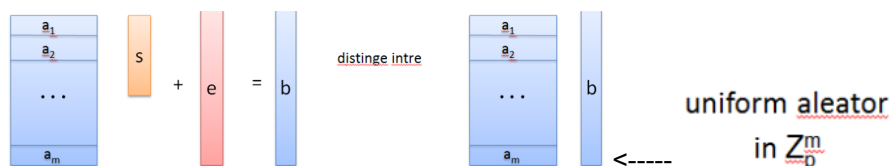
sub forma matriciala

$$\begin{bmatrix} 12 & 10 & 5 & 2 \\ 3 & 7 & 9 & 1 \\ 16 & 2 & 8 & 7 \end{bmatrix} \cdot \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix} \approx \begin{bmatrix} 8 \\ 4 \\ 3 \end{bmatrix}$$

sau, notand matricile coresp. (unde $\mathbf{s} = (s_1, s_2, s_3)$), ec. matriciala devine $\mathbf{A}\mathbf{s} + \mathbf{e} = \mathbf{b}$ mod q

- vectorul $\mathbf{e} = (1, -1, 2)$ form. din elem. "mici" din $\mathbb{Z}$ numite *noise* sau *error*
- In lipsa lui $\mathbf{e}$, ec. $\mathbf{A}\mathbf{s} = \mathbf{b}$ devine usor de rez. cu tehn. clasice de alg. lin.
- Cand mat. $\mathbf{A}$ are suficient de multe linii si param. sunt alesi coresp., pb. devine dificila

Problema decizionala LWE



- Pb decizionala cere s se distinga intre un $\mathbf{b}$ generat ca mai sus (stanga) si un $\mathbf{b}$ generat uniform aleator in $\mathbb{Z}_q^m$

Sistem de criptare bazat pe LWE

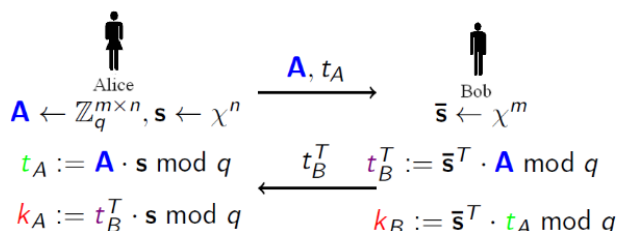
Descriem mai intai un schimb de chei (nesigur) care poate fi vazut ca o versiune algebric liniara a schimbului de chei Diffie-Hellman. Fixeaza param.

$$n, q, \chi, m > n$$

- Verificam usor ca Alice si Bob parajeara aceeasi cheie

$$k_A := t_B^T \cdot \mathbf{s} = \bar{\mathbf{s}}^T \cdot \mathbf{A} \cdot \mathbf{s} = \bar{\mathbf{s}}^T \cdot t_A = k_B$$

- Protocolul de mai sus nu e sigur pt ca un atacator poate calc. $\mathbf{s}$ negat sau $\mathbf{s}$ cu notiuni de algebra liniara si poate afla si cheia



- Insa el poate fi transf. intr-un protocol sigur si adaptat ca un sistem de criptare adaugand "noise", sub ipoteza pb. decizionale LWE.

Schema de mai sus cripteaza un bit iar decriptarea se face calculand $x = v - u \cdot s$.

Rezultatul va fi 1 daca x este mai aproape de q/2 decat de 0.

Apropierea lui x de q/2 este verif.

calculand valoarea absoluta a lui $(x - q/2)$ mod q

Sistem de criptare bazat pe LWE - securitate

- Decriptarea functioneaza corect cat timp

$$|\bar{s} \cdot e + e_2 - e_1 \cdot s| < (q - 1)/4$$

- Aceasta conditie este indeplinita daca distributia χ a valorilor s, s negat, e, e₁, e₂ este aleasa corect si produce nr intregi suficient de mici.

- Sist. de criptare este CPA-sigur (chiar si pt adversari cuantici) daca pb. decizionala LWE este dificila.

Exercitii

Fie El Gamal cu $pk = (g, h = g^a)$ și $sk = (g, a)$ în $\mathbb{G}$.

- **Enc:** data o cheie publica $(\mathbb{G}, q, g, h)$ si un mesaj $m \in \mathbb{G}$

alege $y \leftarrow^R \mathbb{Z}_q$ și întoarce $c = (c_1, c_2) = (g^y, m \cdot h^y)$

- **Dec:** data o cheie secreta $(\mathbb{G}, q, g, a)$ si un mesaj criptat $c = (c_1, c_2)$, întoarce $m = c_2 \cdot c_1^{-a}$

Vrem sa distrib. cheia secreta la 2 pers. a.i. numai ele impreuna pot decripta. Un mod simplu de a rez. aceasta pb. e sa alegem 2 nr. aleat. $a_1, a_2 \in \mathbb{Z}_n$ asa incat $a_1 + a_2 = a$. O pers. ia a_1 , iar cealalta a_2

Pentru decriptarea (c_1, c_2) trimitem c_1 ambelor persoane.

Ce val. trb sa calc. cele 2 pers. si sa ne trimita inapoi a.i. sa putem decript. textul criptat trimis?

Solutie: Pers. 1 trimite $u_1 = c_1^{a_1}$ iar pers. 2 trimite $u_2 = c_1^{a_2}$. Produsul $u_1 \cdot u_2 = c_1^{a_1+a_2} = c_1^a$ impreuna cu c_2 poate fi folosit la decriptare.

Se considera $H: \{0, 1\}^* \rightarrow \{0, 1\}^2$ o fct hash rezist. la coliz. si la a doua preimagine. Se def. o fct.

$H': \{0, 1\}^* \rightarrow \{0, 1\}^{n+1}$ astfel:

Arg. ca H' este rezist. la coliziuni.

Solution: Fie $H'(x_1) = H'(x_2)$ cu $x_1 \neq x_2$.

Daca $H'(x_1) = x_1 || 1 \Rightarrow x_1 = x_2$ contradictie.

Daca $H'(x_1) = H(x_1) || 0 = H(x_2) || 0$ atunci se determ. o coliz. pt $H \Rightarrow$ contradictie

$$H'(x) = \begin{cases} x || 1 & \text{dacă } x \in \{0, 1\}^n \\ H(x) || 0 & \text{altfel} \end{cases}$$

Fie $(Mac, Vrfy)$ un MAC sigur def. peste (K, M, T) unde $M = \{0, 1\}^n$ si $T = \{0, 1\}^{128}$. Este MAC-ul din dreapta sigur? Argumentati raspunsul.

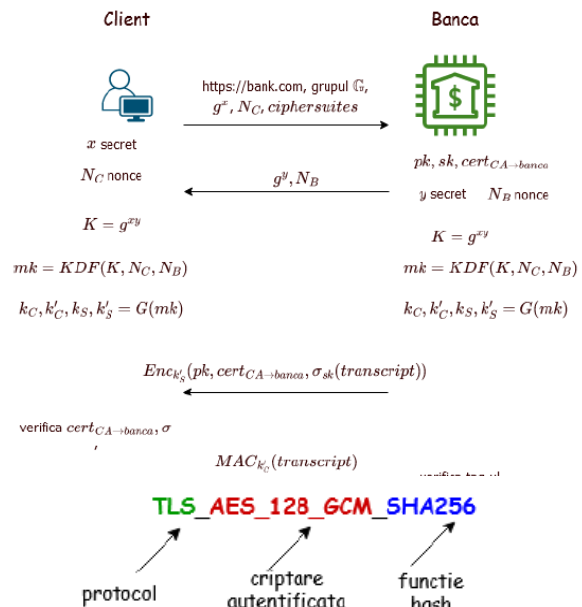
Solution: MAC-ul nu e sigur pt ca un adversar poate sa intoarca perechea valida $(0^n, 0^s)$.

$$\begin{aligned} Mac'(k, m) &= Mac(k, m) \\ Vrfy'(k, m, t) &= \begin{cases} Vrfy(k, m, t), & \text{dacă } m \neq 0^n \\ 1, & \text{altfel} \end{cases} \end{aligned}$$

Curs 13 - TLS (Transport Layer Security)

- Folosit de https
- Numit initial SSL - Secure Sockets Layer, dezvoltat de Netscape 1995 - SSL 3.0 cel mai cunoscut
- TLS 1.0 (1999), TLS 1.1 (2006), TLS 1.2 (2008)
- TLS 1.3 (2018) - actual, sigur si eficient
- Folosirea SSL 3.0, TLS 1.0 si 1.1 trebuie evitată deoarece sunt nesigure

- TLS permite unui client (ex browser) si unui server (ex website) sa se puna de acord asupra unui set de chei pe care sa le foloseasca ulterior pt autentificare
- Are 2 parti
 - 1. Protocolul handshake - realizeaza schimbul de chei care stabileste un set de chei comune
 - 2. Protocolul record-layer - foloseste cheile stabilite pentru criptare/autentificare ulterioara
- TLS 1.3:
 - KDF - key derivation function, algoritm criptografic pt deriv. de chei pe baza unui PRF
 - G - generator de numere pseudo-aleatoare
 - Transcript - reprezinta mesajele transmise in cadrul prot. pana la momentul curent
 - Ciphersuites - colectie de algoritmi criptografici
 - Cheile K'_s si k'_c sunt folosite doar in handshake
 - TLS 1.3 suporta 5 ciphersuites
 - + TLS_AES_128_GCM_SHA256
 - + TLS_AES_256_GCM_SHA384
 - + TLS_CHACHA20_POLY1305_SHA256
 - + TLS_AES_128_CCM_SHA256
 - + TLS_AES_128_CCM_8_SHA256



Alte informatii TLS 1.3

- Clientul si serverul partajeaza la finalul protocolului cheile k_s si k_c pe care le folosesc ulterior pentru autentificare si criptare
- Clientul are garantia ca la final a partajat cheile cu server-ul legitim si ca acestea nu au fost interceptate sau modificate de o terta parte
- In handshake-ul TLS1.2 clientul si serverul foloseau o cheie de criptare cu cheie publica in locul schimbului de chei Diffie-Hellman; clientul doar alegea o cheie K pe care o trimitea serverului criptata cu cheia lui publica
- Aceasta a fost eliminata in TLS 1.3 pentru ca violeaza *forward secrecy* - in cazul compromiterii cheile de sesiune anterioare nu sunt compromise
- In TLS 1.3 in ceea ce priveste server-ul cheia K este calculata pe baza secretului y in fiecare sesiune, secret care este unul nou de fiecare data si care nu trebuie mentinut intre sesiuni => compromiterea serverului nu duce la aflarea cheii
- In TLS 1.2 compromiterea cheii secrete K duce la compromiterea tuturor cheilor din toate sesiunile
- In protocolul *record* clientul si serverul comunica in mod confidential si autentificat folosind o schema de criptare autentificata

TLS 1.3 vs 1.2

- Tls 1.3 are runde mai putine in handshake - 1.2 avea o runda extra in care doar trimitea Hello, acum se in prima runda Hello in timp ce se face si schimbul de chei
- Au fost eliminati algoritmi criptografici vulnerabili - SHA-1, RC4, DES, 3DES, AES-CBC, MD5
- 0-RTT (zero round-trip) - reluarea unei sesiuni mai vechi cu un website e mult mai rapida